WHO KNOWS...
THE GAME IS THE GAME

A REPORT SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF BACHELOR OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

Student id: 11372282

Department of Computer Science

Contents

Word Count: 20830

List of Tables

List of Figures

Abstract

Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

i. The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the "Copyright") and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii. Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the "Intellectual Property") and any reproductions of copyright works in the thesis, for example graphs and tables ("Reproductions"), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv. Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420`), in any relevant Thesis restriction declarations deposited in the University Library, The University Library's regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University's policy on presentation of Theses

Acknowledgements

I would like to thank Messier Bob Marley...

Todo list

- 3_4_snn_architecture.tex §Backward Pass — strip the opening "dead neuron" motivation paragraph (covered in Sections 2.1 and 2.3.2). Retain only the ArcTangent (ATan) equation and the project-specific α hyperparameter value, with a back-reference to Section 2.3.2. 32
- 3_4_snn_architecture.tex §SEW-ResBlocks — strip the vanishing spike explanation and remove the duplicate sewresblock_diagram.pdf figure (now in Figure 2.1). Reduce the intro to: "as detailed in Section 2.3.3, SEW ResBlocks resolve the vanishing spike problem by placing the residual operation after the spiking activation function [9]". 32
- 3_7_sections/temporal_evidence.tex §Hypothesis: Bounding Box Sharpening — strip the Thorpe rapid categorisation paragraphs (transferred to Section 2.3.4). Reduce to one sentence establishing the sharpening hypothesis, then proceed directly to the experimental methodology. 32
- 3_3_data_preprocessing.tex intro paragraphs 1–2: the event format definition transfers to Section 2.4.1. Reduce §3.3 intro to: "Raw event data takes the form $e = (x, y, t, p)$, as defined in Section 2.4.1. The raw ETraM data was provided as pre-recorded .h5 event files at 1280×720 spatial resolution." 35
- 3_3_data_preprocessing.tex §Spatiotemporal Voxelization: the three-bullet conceptual description of temporal binning, polarity splitting, and event accumulation transfers to Section 2.4.2. Retain only the REPLICA-specific parameters ($T = 10$, resulting tensor shape [10, 2, 720, 1280]) with a back-reference to Section 2.4.2 for the mechanism. 35
- 3_7_sections/dynamic_power.tex opening paragraph: strip the MAC static computation explanation and the MAC/AC energy definitions (now in Sections 2.1 and 2.6.2). Retain only the SpikeRateMonitor mechanism and the project-specific ζ derivation, with back-references to Sections 2.1 and 2.6. 38
- 3_7_sections/energy_consumption.tex: strip the $E_{\text{CNN}}/E_{\text{SNN}}$ formula derivations and the Horowitz energy constants preamble (now in Section 2.6). Retain only the platform extrapolation results, the Energy Savings formula, and the results table, with back-references to Section 2.6. 38
- Uncomment line once references are resolved 42
- Check & Proof-read this section against rubric. 42
- Re-write this section, and link it to the Success Criteria from Introduction. . 44
- Add model architecture diagram. 60
- Check & Add model hyperparameter table. 60

List of Abbreviations

AC Conditional Addition. 1, 3, 19

ADAS Advanced Driver Assistance Systems. 1, 18, 19, 21, 22

ADP Average Detections Per Frame. 1

AMP Automatic Mixed Precision. 1

ANN Artificial Neural Network. 1, 25, 26, 29, 30

ATan ArcTangent. 1, 13, 31, 36

BAF Background Activity Filtering. 1

BPTT Backpropgation Through Time. 1, 31

CIoU Complete Intersection over Union. 1

CMOS Complementary Metal-Oxide-Semiconductor. 1

CNN Convolutional Neural Network. 1, 2, 6, 7, 18–21, 23, 24, 26, 27, 30, 35

CPU Central Processing Unit. 1

CUDA Compute Unified Device Architecture. 1, 36

ECE Expected Calibration Error. 1, 21, 23

ETraM Event-based Traffic Monitoring Dataset. 1, 3, 13, 18, 20–22, 24

FLOP Floating-Point Operation. 1, 18, 27

FPS Frames Per Second. 1, 23, 37

FRR Firing Rate Regularization. 1, 35

GIoU Generalised Intersection over Union. 1, 22, 28, 29

GPU Graphics Processing Unit. 1, 20, 21, 23, 36

HD High Definition. 1, 20

HPC High Performance Computing. 1

IBM International Business Machines. 1

IoU Intersection over Union. 1, 21, 22, 28, 29

LIF Leaky Integrate-and-Fire. 1, 4, 5, 12, 24, 29, 30, 36

LRU Least Recently Used. 1

MAC Multiply-Accumulate. 1–3, 19, 23, 26, 27

mIoU Mean Intersection Over Union. 1, 21

ReLU Rectified Linear Unit. 1, 25, 30

ResNet Residual Neural Network. 1, 31

ROC Rank Order Coding. 1, 36

SE Squeeze-and-Excitation. 1, 33, 34

SEW Spike-Element-Wise. 1, 7, 13, 31, 32, 36

SNN Spiking Neural Network. 1, 6, 7, 18–21, 23, 24, 26, 29–31, 33, 35, 36, 39

SNR Signal-to-Noise Ratio. 1

SOP Synaptic Operation. 1, 21, 23

TTFS Time-to-First-Spike. 1, 36

WCET Worst-Case Execution Time. 1

Chapter 1

Introduction

1.1 Overview

Advanced Driver Assistance Systems (ADAS) and self-driving vision systems are computer-vision based solutions implemented by the road cars of today, mandated both by consumers and legislation bodies. Consequently, autonomous perception currently represents one of the most critical frontiers of development in automotive engineering and research. However, the pursuit of full autonomy has driven a relentless escalation in the complexity and resolution of these perception systems: modern vehicles rely on Convolutional Neural Networks (CNNs) to process high-definition video streams in real-time, executing billions of dense, Floating-Point Operations (FLOPs) every second.

As the industry scales towards higher resolutions and faster reaction times to ensure passenger safety, the energy demands of this continuous, dense spatial reasoning have begun to push the limits of what resource-constrained edge environments can sustain [21], posing both a hardware problem to develop more compute-dense computers and a software problem to develop more efficient algorithms.

This project investigates utilizing cutting-edge neuromorphic paradigms, more specifically Spiking Neural Networks (SNNs), and their feasibility in this domain as an alternative computational approach. To achieve this, the project develops `DetectorNX ULTRA`, a high-definition spiking architecture designed for multi-object automotive perception. By rigorously benchmarking this system against a strictly equivalent continuous CNN on the event-based Event-based Traffic Monitoring Dataset (ETraM) dataset [38], this research empirically deconstructs the performance-efficiency frontier—quantifying the precise trade-offs between the energy savings of event-driven dynamics and the spatial precision constraints of 1-bit temporal quantization.

1.2 Motivation

The primary motivation for this research is rooted in the severe thermal and power consumption trade-offs inherent to deploying continuous-valued Convolutional Neural Networks (CNNs) in autonomous edge environments. As established by Lin et al. [21], the deployment of deep learning models on resource-constrained Advanced Driver Assistance Systems (ADAS) platforms exposes a critical sustainability issue: the static, frame-based nature of CNNs mandates that they execute billions of dense MAC operations continuously, generating immense metabolic load and thermal throttling even when processing visually redundant or static environments.

To resolve these computational and thermal constraints, this research looks toward the most power-efficient perception engine known: the biological brain. Unlike traditional cameras that capture absolute frames at a fixed frequency, biological retinas and neuromorphic event sensors only transmit asynchronous changes in luminance. Consequently, biological neural networks consume metabolic energy only when processing new, relevant information [25].

Spiking Neural Networks (SNNs) are the computational embodiment of this biological sparsity. By replacing continuous, energy-heavy MAC operations with discrete, event-driven Conditional Addition (AC) additions, SNNs offer a theoretical pathway to sub-milliwatt, real-time object detection at the edge. Since spiking neurons remain dormant until triggered by an input event and respond to a stimulus, they naturally bypass the thermal throttling and battery drain issues that plague continuous CNNs [21].

However, a significant gap exists in the current literature regarding the deployment of SNNs for high-definition automotive perception: the transition from 32-bit floating-point CNN activations to 1-bit discrete spikes introduces quantization errors and temporal precision limits. Historically, this has restricted SNNs to low-resolution classification tasks, as they struggle to resolve the fine-grained spatial coordinates required for multi-object bounding box regression in complex traffic scenes.

The `DetectorNX` project is motivated by the critical need to both rigorously quantify and mitigate these trade-offs on modern, high-definition event datasets. By establishing strict architectural parity between an SNN and a CNN baseline, this project seeks to definitively prove whether the immense thermal and power savings of the neuromorphic paradigm can be achieved without fatally compromising the spatial precision required for autonomous vehicle safety.

1.3 Project Aims and Objectives

This project pursues a single, well-defined research question: can a SNN, constrained to the same architectural capacity as a continuous CNN, achieve competitive object detection accuracy on high-definition event-based data while delivering meaningful energy efficiency gains? To answer this, the following objectives were established:

1. **Develop a high-definition SNN for multi-object detection:** Design and train `DetectorNX ULTRA`, a spiking architecture capable of performing bounding box regression on the High Definition (HD) ETraM event-camera dataset.
2. **Develop a structurally equivalent CNN baseline:** Construct `CNN_MAX_REPLICA_SE`, a continuous-valued counterpart that mirrors the SNN architecture layer-for-layer with an identical parameter budget, isolating the spiking mechanism as the sole independent variable in all subsequent experiments and evaluations.
3. **Train and evaluate both models under identical conditions:** Apply the same pre-processed dataset, loss function, optimiser, and training duration to both models, ensuring that any observed performance differences are attributable solely to the computational paradigm.
4. **Benchmark inference latency on Graphics Processing Unit (GPU) hardware:** Measure the real-time throughput of both architectures to assess their viability for deployment in automotive perception pipelines.
5. **Estimate theoretical energy consumption:** Derive energy usage from layer-wise spike activity using established neuromorphic hardware constants, and project consumption figures onto representative neuromorphic platforms such as Intel Loihi and IBM TrueNorth.
6. **Conduct a rigorous, multi-dimensional comparison:** Synthesise the accuracy, latency, and energy results to empirically characterise the performance-efficiency frontier of the spiking paradigm on a real-world, high-definition detection task.

1.4 Success Criteria

To provide a concrete evaluation framework, the following measurable success criteria were defined prior to experimentation. Each criterion is revisited explicitly in Chapter 4.

1. **Detection accuracy parity:** The SNN must achieve a Mean Intersection Over Union (mIoU) of at least 90% of the CNN baseline's score on the ETraM validation set. Achieving this will validate that our architecture succeeds in mitigating the compromises expected by the 1-bit temporal quantisation error.

2. **Measurable theoretical energy savings:** The SNN must demonstrate a reduction in theoretical energy consumption per inference relative to the CNN baseline, computed from layer-wise Synaptic Operation (SOP) counts and published neuromorphic hardware constants. The reduction in energy consumption must be realised on real GPU hardware, even if it is not to the same extent as the extrapolated values on neuromorphic hardware.

3. **Real-time viability:** Both models must sustain an inference throughput of at least 30 frames per second on GPU hardware, meeting the established minimum threshold for real-time automotive perception [19].

4. **Architectural parity:** The SNN and CNN must share an equivalent parameter count (within 1%), ensuring that any observed performance gap reflects the spiking mechanism rather than differences in model capacity.

5. **Detection completeness:** Both models must achieve a recall of 100% for objects with an Intersection over Union (IoU) exceeding 0.5 on the validation set, confirming that the multi-box regression head reliably detects all ground-truth vehicles in the scene.

6. **Confidence calibration:** The SNN must exhibit confidence calibration equivalent to, or superior to, that of the CNN baseline, as measured by the Expected Calibration Error (ECE) computed from reliability diagrams on the ETraM validation set. This criterion is safety-critical: in ADAS applications, a model that systematically over-states its detection confidence may suppress downstream safety interventions. A neuromorphic architecture that delivers energy savings at the cost of increased over-confidence would therefore be unsuitable for deployment, regardless of its accuracy or efficiency gains.

1.5 Evaluation Approach

Evaluation in this project is designed to isolate the effect of the spiking mechanism as precisely as possible. Because `DetectorNX ULTRA` and `CNN_MAX_REPLICA_SE` are

constrained to within 1% of each other in parameter count and are trained under strictly identical conditions, any observed differences in the metrics described below can be attributed to the computational paradigm rather than to capacity or training asymmetries.

A validation set is created from the original ETraM dataset for all experiments, which was built using a stratified sampling approach further described in Section **??**, to ensure that all results obtained were collected across all available environment scenarios: night, day, rain, perfect weather and more.

1.5.1 Prediction Performance

To evaluate both models' performance at predicting the position of vehicles across the event-driven scenes, the following metrics will be employed:

1. **Intersection over Union (IoU):** The primary measure of detection quality is mean IoU, aggregated across all predictions on the held-out validation split. Standard IoU quantifies the ratio of the intersection to the union of predicted and ground-truth bounding boxes, providing a scale-invariant, directly interpretable measure of spatial overlap that is universally comparable across architectures and datasets. It is important to note the distinction between the evaluation metric and the training loss: during training, Generalised Intersection over Union (GIoU) is employed as the loss function because standard IoU produces zero gradients when predicted and ground-truth boxes do not overlap, stalling optimisation [27]. Generalised Intersection over Union (GIoU) resolves this by incorporating a penalty term derived from the smallest enclosing convex hull, providing a meaningful gradient signal even in the non-overlapping case. For evaluation, however, standard IoU is used in preference to GIoU because the additional geometric term in GIoU is a training convenience rather than an interpretable measure of detection quality, and because standard IoU enables direct comparison with published results in the event-based detection literature.

2. **Recall:** Detection completeness is assessed by computing recall at an IoU threshold of 0.5 across the full validation set. In safety-critical ADAS applications, a missed detection is categorically more dangerous than a false positive: an undetected vehicle cannot be acted upon, whereas a spurious detection can be suppressed by downstream filtering. A recall of 100% at this threshold is therefore treated as a hard deployment requirement and evaluated as a binary pass or fail criterion rather than a continuous metric.

3. **Expected Calibration Error (ECE):** Confidence calibration is assessed by computing the Expected Calibration Error (ECE) from reliability diagrams generated on the validation set, following the methodology of Guo et al. [13]. A well-calibrated model produces confidence scores that faithfully reflect empirical detection accuracy: a prediction assigned 80% confidence should be correct approximately 80% of the time. This criterion is particularly relevant when comparing the SNN against the CNN, as the temporal quantisation inherent to spike-based computation may introduce systematic over- or under-confidence that would be invisible to accuracy metrics alone. In an autonomous perception pipeline, a systematically over-confident model may suppress safety-critical interventions; the ECE makes this failure mode directly measurable.

1.5.2 Model Performance

To evaluate both models' suitability for real-world deployment, two complementary efficiency dimensions are measured:

1. **Inference Latency:** Throughput is measured by timing forward passes over a fixed batch of validation frames on the same GPU hardware, reporting mean latency in milliseconds per frame and the equivalent Frames Per Second (FPS). The 30 FPS threshold established by Khan, Rizvi and Dengel [19] as the minimum for real-time automotive perception is used as the viability boundary. This measurement is conducted on GPU hardware rather than neuromorphic hardware, since no physical neuromorphic device was available for this project.

2. **Theoretical Energy Consumption:** Energy consumption is estimated analytically rather than measured on physical hardware. Each layer's energy footprint is computed from its operation count: MAC operations for the CNN and Synaptic Operation (SOP) for the SNN, scaled by published per-operation energy constants (E_{MAC} and E_{AC} respectively) [15]. This approach is chosen because it yields a hardware-agnostic efficiency measure that isolates the effect of the spiking mechanism, independent of implementation-specific factors such as clock frequency or memory bandwidth. Aggregate figures are then projected onto representative neuromorphic targets, Intel Loihi and IBM TrueNorth, to contextualise the savings achievable under neuromorphic deployment.

1.6 Report Structure

In the following chapters, we cover the following:

Background: Chapter 2 provides the theoretical and empirical context required to situate the `DetectorNX` project. It surveys the development of CNNs for object detection, introduces the biological principles underlying SNNs and the LIF neuron model, reviews the event-camera sensing paradigm and the ETraM dataset, characterises the energy economics of neuromorphic hardware, and synthesises related work comparing spiking and continuous architectures on detection tasks.

Re-write this once background is complete.

Methodology: Chapter 3 presents the full technical implementation. It details the system requirements, the shared multi-box detection head architecture, the data preprocessing pipeline, the design of `DetectorNX ULTRA` and `CNN_MAX_REPLICA_SE`, the architectural evolution that led to the final configurations, and the complete experimental setup including training hyperparameters, hardware environment, and evaluation protocol. Results and their analysis are presented within this chapter as an integrated discussion of methodology and outcome.

Conclusions: Chapter 4 concludes the report. It revisits each success criterion defined in Section 1.4, summarises the principal findings across the accuracy, latency, and energy dimensions, reflects critically on the limitations of the experimental design, and proposes concrete directions for future work including deployment on physical neuromorphic hardware and exploration of alternative spiking encodings.

Chapter 2

Background

2.1 Artificial Neural Networks

2.1.1 What is an Artificial Neural Network?

Artificial Neural Networks (ANNs) are a family of computational models loosely inspired by the structure of biological neural tissue [20]. An ANN is composed of layers of interconnected nodes, or neurons, each of which computes a weighted sum of its inputs and passes the result through a non-linear activation function. By stacking multiple of these layers in sequence, the network learns a hierarchical composition of transformations that progressively maps raw input data to a task-specific output representation. The dominant activation function in modern deep learning is the Rectified Linear Unit (ReLU), defined as $f(x) = \max(0, x)$, which is computationally inexpensive and empirically effective at enabling deep networks to learn complex, non-linear functions [20].

2.1.2 Training an Artificial Neural Network

Training an ANN is performed by minimising a differentiable loss function over the training set via gradient descent. Gradients are propagated from the output layer back through the network using the chain rule of calculus, using a procedure known as backpropagation [20]. A fundamental requirement of this process is that every function in the computational graph must be differentiable: if a non-differentiable operation is encountered, the chain rule breaks down and no gradient signal can propagate through it. This constraint is unproblematic for standard ANNs where activations such as ReLU are

differentiable almost everywhere, but becomes a significant challenge for the spiking paradigm, which is non-differentiable, as discussed in Section 2.3.

2.1.3 Multiply-Accumulate Operations

Deep ANNs rely predominantly on Multiply-Accumulate (MAC) operations for their arithmetic procressing: for each neuron, its output is the accumulated product of incoming values and their corresponding weights. In a standard convolutional or fully connected layer, the total MAC count is a fixed function of the architecture, determined by filter dimensions, channel widths, and spatial resolution, which is independent of the input content as a result. This static computational budget is a defining characteristic of synchronous deep learning, and forms the baseline against which the event-driven efficiency of input-dependent SNNs is evaluated in Section 2.6.

§3.4 `Backward Pass` opening paragraph (lines 1–3) re-explains why backpropagation requires differentiability. Once §2.1 is in place, reduce to a single sentence: "since the Heaviside function is non-differentiable (§2.1), conventional backpropagation fails immediately", then proceed to the surrogate solution.

§3.7 `Dynamic Power` opening paragraph re-explains MACs as a static computational primitive. Once §2.1 is in place, reduce the first sentence to a brief reference to §2.1 rather than re-defining MACs inline.

2.2 Convolutional Neural Networks for Object Detection

2.2.1 Convolutional Feature Extraction

A Convolutional Neural Network (CNN) extends the standard ANN architecture by replacing dense linear layers with convolutional operations, in which a bank of learned filters is applied across the spatial extent of an input. Each filter performs a localised weighted sum at every spatial position, producing a feature map that encodes the presence and magnitude of a particular learned pattern at each location. Convolutional layers can also be stacked to build a hierarchy of representations: early layers detect low-level geometric patterns such as edges and motion boundaries, whilst deeper layers synthesise these into semantically meaningful object features [20].

Throughout a stacked network, the spatial resolution is progressively reduced using

strided convolutions or pooling operations, compressing the input into increasingly compact, dense high-level representations.

2.2.2 Calculating the MAC of a CNN Layer

The total number of MAC operations in a single convolutional layer is a fixed function of its geometry, and can be calculated as follows:

$$\text{FLOPs}_l = H_{out} \times W_{out} \times K^2 \times C_{in} \times C_{out} \tag{2.1}$$

where $(H_{out} \times W_{out})$ is the spatial resolution of the output feature map, K is the kernel size, and C_{in}, C_{out} are the input and output channel widths respectively. Subsequently, the equation produces a Floating-Point Operation (FLOP) value, which is the total number of floating point operations, specifically multiplication and addition, performed during the forward pass of the layer. As established in Section 2.1, this count is static, determined entirely by the architecture and independent of the visual content of the input, which is an inherent problem for energy-efficiency in such architectures.

2.2.3 Residual Learning

A well-documented failure mode of deep CNNs is the *vanishing gradient* problem: as backpropagation proceeds through many successive layers, gradients are repeatedly scaled by values less than one, causing them to decay exponentially and preventing effective learning in early layers [14]. He et al. [14] resolved this through the introduction of *residual learning*: rather than tasking each layer with learning a direct mapping $y = \mathcal{H}(x)$, the network instead learns the *residual* $\mathcal{F}(x) = \mathcal{H}(x) - x$, recasting the target as:

$$y = \mathcal{F}(x) + x \tag{2.2}$$

The additive shortcut connection, otherwise known as a skip connection, bypasses one or more convolutional layers and adds the input directly to the output, providing an unobstructed gradient path back to the earliest layers, guaranteeing stable training regardless of network depth. Within a Standard Residual Block, this shortcut carries the raw input features forward through the identity path, and the convolutional branch learns only the incremental refinement needed to improve the representation. This mechanism forms the backbone of the CNN baseline employed in this project and, through its spiking equivalent discussed in Section 2.3, underpins the `DetectorNX`

ULTRA architecture itself.

2.2.4 Object Detection & Bounding Box Regression

Object detection requires a network to simultaneously localise and identify one or more targets within a scene, and determine what its coordinates and size in the space are. In the bounding box regression paradigm popularised by Redmon et al. [26], this is framed as a direct regression problem: the network predicts a set of candidate boxes, each parameterised by a feature tuple $\mathcal{F} = \{x, y, w, h, c\}$, where (x, y) and (w, h) are the normalised centre coordinates and dimensions of the box, and c is an objectness confidence score reflecting the probability that a genuine target is present within that region.

By predicting multiple candidate boxes simultaneously in a single forward pass, this multi-box formulation enables real-time detection across the full spatial extent of the input; in a real-world application, an autonomous vision system must be able to handle detecting multiple vehicles and artifacts simultaneously, making this multi-box paradigm even more prevalent. Each predicted box is evaluated against the ground-truth annotations using an Intersection over Union (IoU) metric, which quantifies the ratio of the intersection to the union of the two boxes, providing a scale-invariant, directly interpretable measure of spatial overlap. The multi-box regression paradigm, including the (x, y, w, h, c) output format and the use of IoU as the evaluation criterion, forms the direct foundation of the detection head employed in this project, as detailed in Section 3.4.5.

2.2.5 GIoU Loss

Whilst standard IoU is an effective evaluation metric, it presents a fundamental problem when used directly as a training loss: when a predicted box and its ground-truth target do not overlap at all, the IoU score is zero regardless of how far apart the two boxes are. Consequently, the gradient of a standard IoU loss is also zero in this case, providing no optimisation signal to guide the network towards the correct region and stalling training in the early stages when predictions are furthest from the ground truth. Rezatofighi et al. [27] resolved this by proposing Generalised Intersection over Union (GIoU), which augments the standard overlap term with a penalty derived from the area of the smallest

enclosing convex hull C that contains both boxes:

$$\text{GIoU} = \text{IoU} - \frac{|C \setminus (A \cup B)|}{|C|} \tag{2.3}$$

This penalty is non-zero even when the boxes are entirely disjoint, ensuring a meaningful gradient signal is always available. As the predicted box converges towards the ground truth, the GIoU penalty vanishes and the loss reduces to standard IoU, making it a strict generalisation of the original metric. GIoU is employed as the training loss throughout this project, with standard IoU being reserved for evaluation, where interpretability and comparability with published results take precedence over gradient behaviour.

3_5_cnn_baseline.tex §Backbone: the two-sentence He et al. / skip connections explanation can be reduced to a single cite pointing here: "The inclusion of skip connections enables the CNN to avoid the vanishing gradient problem [14] (see §2.2)".

3_5_cnn_baseline.tex §SEWResBlock vs Standard ResBlock, CNN-side prose: strip the explanation of what a Standard ResBlock does and retain only the comparison table. The mechanism now lives in §2.2.

3_7_sections/dynamic_power.tex: the inline parenthetical explaining the FLOPs formula variables ("where $H_{out} \times W_{out}$ is the number of pixels...") can be removed — the reader now has the full formula in §2.2.

2.3 Spiking Neural Networks

SNNs represent a fundamentally different computational paradigm to the standard ANN architectures described in the preceding sections. Rather than propagating continuous-valued activations between layers, SNNs communicate exclusively through discrete binary events, spikes, mimicking the all-or-nothing firing behaviour of biological neurons [28]. By prioritising event-driven binary communication, this paradigm aims to drastically reduce both dynamic power consumption and computational complexity compared to traditional deep learning models, making it inherently suited to power and latency-constrained environments.

2.3.1 The Leaky Integrate-and-Fire Neuron

At the foundation of most practical SNNs lies the Leaky Integrate-and-Fire (LIF) neuron model, which governs the temporal dynamics and binary spike generation of

the network. Unlike standard ANNs that utilise continuous activation functions such as ReLU, the LIF node maintains an internal state: the membrane potential $V[t]$, which updates iteratively across discrete timesteps $t \in [1, T]$.

The dynamics of the LIF neuron at a given spatial position and timestep t are governed by the integration of incoming pre-synaptic spatial features $X[t]$ and an inherent leak factor τ, which dictates how quickly $V[t]$ returns to its resting state in absence of incoming input. The hidden membrane potential $H[t]$ prior to spiking is mathematically defined as:

$$H[t] = V[t-1] + \frac{1}{\tau}(X[t] - (V[t-1] - V_{reset})) \tag{2.4}$$

When $H[t]$ exceeds a predefined threshold V_{th}, the neuron emits a binary spike $S[t] \in \{0, 1\}$ to the subsequent layer, governed by the Heaviside step function $\Theta(x)$:

$$S[t] = \Theta(H[t] - V_{th}) \tag{2.5}$$

Following a spike ($S[t] = 1$), the neuron undergoes a hard reset, instantly dropping its potential $V[t]$ to V_{reset}, mimicking the refractory period of biological neurons, preventing infinite spiking and overstimulation [30]. If no spike is emitted ($S[t] = 0$), the potential naturally decays, ensuring that subsequent layers only perform computations upon receiving active events [31]. This *activation sparsity*: the property that computation is only triggered by genuine signal, not by silence, which is the primary driver of the SNN's exceptional energy efficiency [28] — where a CNN triggers computation regardless of the input stimuli, an SNN only triggers synaptic operations in response to discrete spikes, effectively idling during periods of silence.

2.3.2 Surrogate Gradients

Training deep SNNs via backpropagation presents a fundamental mathematical challenge, since the Heaviside step function governing spike generation is non-differentiable. As established in Section 2.1, the chain rule requires every function in the computational graph to be differentiable; the Heaviside function violates this directly, since its derivative is zero everywhere except at the threshold V_{th}, where it is infinite. Using conventional backpropagation therefore results in zeroed gradients that immediately halt the learning process, a phenomenon known as the "dead neuron" problem.

To resolve this, SNNs employ a *surrogate gradient* during the backward pass [24].

While the forward pass remains strictly binary, the backward pass approximates the step function using a smooth, differentiable curve. A common choice is the ATan function:

$$\frac{\partial S}{\partial H} \approx \frac{\alpha}{2\left(1+\left(\frac{\pi}{2}\alpha(H-V_{th})\right)^2\right)} \tag{2.6}$$

where α is a hyperparameter controlling the steepness of the surrogate curve. This mathematical substitution allows the network to learn complex spatiotemporal representations from scratch while maintaining a purely binary forward execution, with gradients propagated through the temporal dimension via Backpropgation Through Time (BPTT).

2.3.3 Spike-Element-Wise Residual Blocks

Building deep SNNs introduces a further challenge beyond the surrogate gradient problem: the *vanishing spike* problem inherent in standard residual connections. In a standard Residual Neural Network (ResNet) (Section 2.2), the residual addition occurs before the activation function. However, applying this directly to a spiking network causes a critical failure, because adding two binary spike trains produces a non-binary, continuous value (e.g., $1+1=2$), which destroys the energy-efficient, event-driven nature of the network [9].

To overcome this, Fang et al. [9] proposed the Spike-Element-Wise (SEW) residual paradigm, instantiated as `SEWResBlock` units. Unlike standard Spiking ResNets, SEW ResBlocks apply the element-wise residual operation after the spiking activation function, guaranteeing that the output remains strictly binary. Fang et al. [9] proved that this reordering is both necessary and sufficient to achieve identity mapping in a purely spike-based network, resolving the vanishing spike problem and enabling representational depth that would otherwise induce spike degradation.

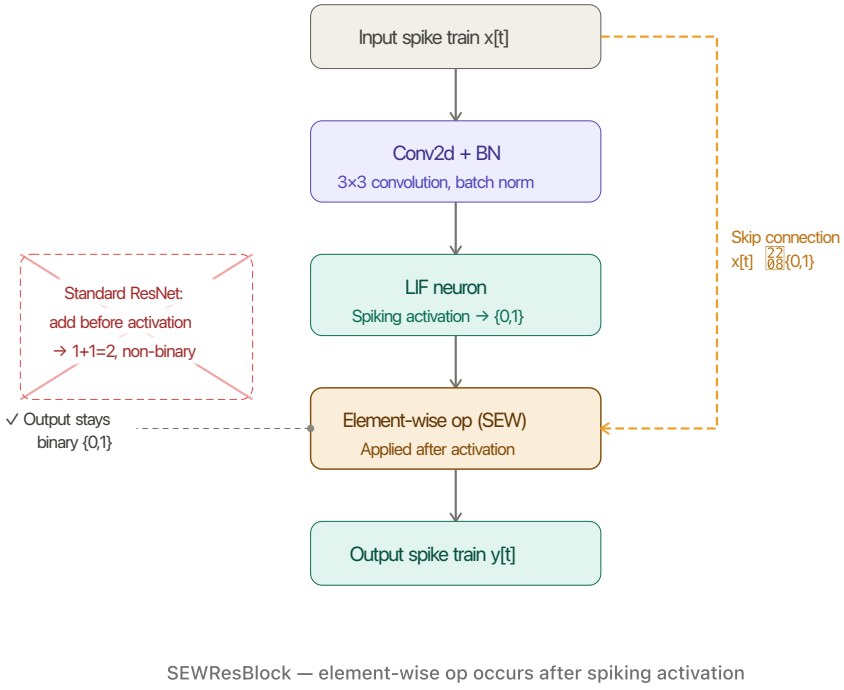


Figure 2.1: The SEW residual block (`SEWResBlock`) as proposed by Fang et al. [9].

2.3.4 Temporal Dynamics & Rapid Categorisation

A defining property of SNNs is their natural ability for spatiotemporal integration, combining timing and location when processing information. By processing an event stream iteratively across T discrete timesteps, an SNN is capable of producing meaningful predictions before the full observation window has elapsed [25]. This property is absent in a standard CNN, which performs a single spatial pass over the fully integrated input and can only produce an output once all temporal evidence has been collected.

This progressive refinement is analogous in spirit to the *rapid categorisation* phenomenon documented in the human visual system, where early feedforward spikes drive coarse semantic decisions that sharpen as further neural evidence accumulates [35]. While Thorpe, Fize and Marlot [35] address categorical recognition rather than coordinate regression, the underlying principle that sparse early spike activity produces meaningful but imprecise representations which sharpen over time is directly consistent with the anticipated behaviour of a spiking detection backbone.

Can refer to this in 3.7.4

Rate coding over a fixed window of T timesteps carries an inherent representational limit known as the ”quantisation error”; the precision of the encoded signal is bounded by a $1/T$ quantisation floor, since information is encoded as spike counts that can take only $T+1$ discrete values [34]. Alternative temporal coding schemes have been proposed to overcome this ceiling, including Time-to-First-Spike (TTFS) encoding, where information is represented by the arrival time of the first spike rather than the firing frequency, and Rank Order Coding (ROC), which represents signal intensities via relative firing orders and sub-millisecond arrival times [34].

`3_4_snn_architecture.tex` §Neuron Dynamics: LIF — strip all equations and mechanistic explanation. Retain only project-specific parameters ($V_{th} = 0.2$, hard reset, τ) and add a back-reference: “the LIF model, as defined in Section 2.3.1, governs the temporal dynamics of the network”.

`3_4_snn_architecture.tex` §Backward Pass — strip the opening “dead neuron” motivation paragraph (covered in Sections 2.1 and 2.3.2). Retain only the ATan equation and the project-specific α hyperparameter value, with a back-reference to Section 2.3.2.

`3_4_snn_architecture.tex` §SEW-ResBlocks — strip the vanishing spike explanation and remove the duplicate `sewresblock_diagram.pdf` figure (now in Figure 2.1). Reduce the intro to: “as detailed in Section 2.3.3, SEW ResBlocks resolve the vanishing spike problem by placing the residual operation after the spiking activation function [9]”.

3_7_sections/temporal_evidence.tex §Hypothesis: Bounding Box Sharpening — strip the Thorpe rapid categorisation paragraphs (transferred to Section 2.3.4). Reduce to one sentence establishing the sharpening hypothesis, then proceed directly to the experimental methodology.

2.4 Event-Based Vision

2.4.1 Event Cameras

Frame-Based vs. Event-Based Sensing

Conventional frame-based cameras operate synchronously, where every pixel is read out at a fixed rate, producing a dense intensity matrix regardless of whether anything in the scene has changed since the last frame. This design introduces several well-documented limitations: motion blur at high speeds, a fixed dynamic range bounded by the exposure window, and substantial data redundancy during static scenes — all of which are particularly problematic for high-speed automotive perception [11].

An event camera fundamentally departs from this paradigm; rather than reading out all pixels simultaneously, each pixel operates independently, firing an event the moment its log-intensity changes beyond a threshold. The result is a continuous, asynchronous stream of discrete events rather than a sequence of synchronous frames, registering only what has changed between scenes instead of the full scenes themselves. In other words, if the scene was completely dormant with no changes occurring, an event-driven camera would capture this as a blank image versus a frame-based camera capturing the empty scene.

Advantages and the Compatibility Problem

This per-pixel, threshold-driven design gives event cameras a compelling profile for autonomous perception: high temporal resolution, high dynamic range, low latency, and low data redundancy [11]. However, a sparse, unordered point cloud of (x, y, t, p) tuples is fundamentally incompatible with conventional deep learning frameworks, which expect dense, structured tensors of fixed spatial dimensions. Correcting this to adapt to deep learning approaches requires a preprocessing step to convert the asynchronous stream into a format suitable for neural network ingestion, which is the function of `spatiotemporal voxelization`.

2.4.2 Spatiotemporal Voxelization

The Voxelization Process

To make the asynchronous event stream compatible with neural network layers, a voxelization process transforms the continuous point cloud into a structured tensor over a temporal window of duration ΔT. This process consists of three core operations:

- **Temporal Binning:** The total duration of the window ΔT is divided into T discrete time bins. Each event is assigned to the bin corresponding to its timestamp t, discretising the continuous temporal axis into T ordered slices.
- **Polarity Splitting:** Events are separated into two distinct channels ($C = 2$) based on their polarity, representing positive and negative brightness changes independently. This preserves the directional information of the log-intensity change, which would be lost if both polarities were accumulated into a single channel.
- **Event Accumulation (Histogramming):** Within each discrete temporal bin and polarity channel, the individual asynchronous events are aggregated at their respective spatial coordinates (x, y). The value of each "pixel" in the resulting spatial grid becomes the total count of events of that polarity that occurred at that location during that fraction of time, effectively converting the sparse point cloud into a sequence of dense 2D event histograms.

This accumulation process produces a dense 4D spatiotemporal tensor of shape $[T, 2, H, W]$ (Time $\times$ Channels $\times$ Height $\times$ Width) that standard convolutional and spiking layers can natively process.

The Temporal Window

The choice of window duration ΔT is a fundamental design parameter that governs the temporal resolution of each voxelized sample. Too short a window results in insufficient event density: if too few events accumulate within a bin, the resulting histograms are near-empty, providing insufficient input signal for the network to extract meaningful features. Too long a window introduces a different failure mode: fast-moving objects displace significantly within the window, causing the bounding box annotation assigned at the window's centre timestamp to become spatially misaligned with the event distribution by the window's end. The specific window duration adopted in this project, and the reasoning behind its selection, is detailed in Section 3.3.

2.4.3 The Natural SNN–Event Camera Pairing

An Event-Driven Architecture for an Event-Driven Sensor

Both systems are fundamentally event-driven: an event camera outputs discrete binary events triggered by scene changes, whilst an SNN processes discrete binary events in the form of spikes triggered by membrane threshold crossings. In both cases, silence is the default operating state, and meaningful activity is generated only in response to genuine stimuli.

A CNN must collapse the sparse, asynchronous event stream into a dense synchronous tensor before it can process it, discarding the temporal structure encoded in the event timestamps in the process. An SNN however processes the voxelized stream iteratively across T timesteps via its LIF dynamics, where each timestep exposes the network to a temporal slice of the event stream, which the LIF neurons integrate progressively into their membrane potentials. This directly mirrors the camera's temporal output structure, preserving and exploiting the event-driven dynamics rather than collapsing them into a single static representation [11, 25].

Implications for Neuromorphic Deployment

On neuromorphic hardware such as Intel Loihi [4] or IBM TrueNorth [22], this alignment extends to the physical level. An event camera connected directly to a neuromorphic chip can route events to spiking neurons without any voxelization or dense tensor conversion, since the raw events can be routed directly to spiking neurons without any intermediate tensor representation, enabling a fully asynchronous end-to-end inference pipeline and eliminating the preprocessing bottleneck entirely.

`3_3_data_preprocessing.tex` intro paragraphs 1–2: the event format definition transfers to Section 2.4.1. Reduce §3.3 intro to: "Raw event data takes the form $e = (x, y, t, p)$, as defined in Section 2.4.1. The raw ETraM data was provided as pre-recorded `.h5` event files at 1280×720 spatial resolution."

`3_3_data_preprocessing.tex` §Spatiotemporal Voxelization: the three-bullet conceptual description of temporal binning, polarity splitting, and event accumulation transfers to Section 2.4.2. Retain only the `REPLICA`-specific parameters ($T = 10$, resulting tensor shape `[10, 2, 720, 1280]`) with a back-reference to Section 2.4.2 for the mechanism.

2.5 ETraM Dataset

2.5.1 Dataset Overview & Acquisition Setup

ETraM is a first-of-its-kind, fully event-based traffic monitoring dataset introduced by Verma et al. [38] at CVPR 2024. Data was captured using the Prophesee EVK4 HD event camera at 1280×720 spatial resolution with a temporal resolution exceeding 10,000 fps, mounted statically at approximately 6 m height with a 35° pitch angle. Unlike ego-motion datasets, where the camera itself moves through the scene, this dataset provides a static perspective; a fixed vantage point ensures that events are generated predominantly by moving traffic participants rather than background infrastructure. Recordings span intersections, roadways, and local streets around Arizona State University, Tempe campus, totalling ten hours of annotated event data.

2.5.2 Class Taxonomy, Scale & Annotation

ETraM covers eight distinct object classes organised into three broad categories: Vehicles (Car, Truck, Bus, Tram), Pedestrians, and Micro-mobility (Bicycle, Bike, Wheelchair). Over 2M bounding boxes were manually annotated using CVAT [38], with each annotation additionally carrying an object ID to support multi-object tracking. Raw data is distributed in multiple formats (RAW, DAT, and H5), with ground-truth annotations provided in NumPy format. For this project, the dataset was abstracted to the Vehicle category exclusively, allowing the SNN–CNN comparison to be isolated on a single, well-defined object class.

2.5.3 Suitability for This Project

Two properties motivated the selection of ETraM for this project. First, critically for an ADAS and autonomous driving context, recordings span the full spectrum of real-world operating conditions: daytime, nighttime, and twilight, across sunny, overcast, and rainy weather - precisely the range of environments an autonomous perception system must handle reliably, and one that deliberately exercises the conditions under which frame-based sensors are known to degrade. Second, the ETraM study provides a dataset-specific spatiotemporal filtering and denoising pipeline tuned to the noise characteristics of the EVK4 sensor, which was adopted directly as the signal conditioning stage of the `REPLICA` preprocessing system, This eliminated the need to develop bespoke noise

suppression from first principles, and significantly accelerated the transition to model development. The full REPLICA pipeline is detailed in Section 3.3.

2.6 Neuromorphic Hardware & Energy Modelling

2.6.1 Neuromorphic Hardware Platforms

Intel Loihi [4] and IBM TrueNorth [22] represent the two principal reference platforms for neuromorphic computing. Unlike conventional GPUs, which execute computation synchronously across all neurons at every clock cycle, both architectures operate asynchronously and event-driven. A neuron consumes energy only when it receives a spike and crosses its firing threshold, remaining entirely silent, drawing no dynamic power in the absence of input activity. This mirrors the SNN's spike-based dynamics precisely, making neuromorphic hardware the natural target deployment environment for event-driven architectures and completing the end-to-end alignment with the event camera described in Section 2.4.3. The per-SOP energy costs of these platforms, 23.6 pJ for Intel Loihi [4] and 26.0 pJ for IBM TrueNorth [22], form the hardware extrapolation targets used in the energy evaluation of this project.

2.6.2 The MAC vs AC Energy Distinction

The foundational energy analysis underlying this project rests on the silicon-level operation costs documented by Horowitz [15]. A standard CNN performs a full MAC at each synaptic connection - a floating-point multiplication followed by an accumulation, costing $E_{\text{MAC}} = 4.6\,\text{pJ}$ per 32-bit operation. An SNN, by contrast, communicates exclusively via binary spikes; since a spike is either 0 or 1, the multiplication is eliminated entirely, reducing each synaptic event to an AC operation at a cost of $E_{\text{AC}} = 0.9\,\text{pJ}$ - a $5\times$ reduction in per-operation energy at the silicon level [15]. This asymmetry is the first of two complementary mechanisms that underpin the SNN's energy advantage over its CNN counterpart.

2.6.3 The Synaptic Operations Metric & Energy Model

The Synaptic Operation (SOP) is the SNN equivalent of the CNN's FLOP, reflecting the number of binary synaptic accumulations performed during a forward pass per layer. Crucially, unlike FLOPs, which are a static function of architecture geometry

that is entirely independent of input content, SOPs are input-dependent, scaled by the activation sparsity ζ of each layer and the number of temporal timesteps T:

$$\text{SOPs}_l = \text{FLOPs}_l \times \zeta_l \times T \tag{2.7}$$

This is the second complementary efficiency mechanism: if the network's average firing rate is low, the SNN performs only a small fraction of the operations that the CNN executes unconditionally for every input. The total energy models for both architectures follow directly from these definitions:

$$E_{\text{CNN}} = \sum_{l=1}^{L} (\text{FLOPs}_l \times E_{\text{MAC}}) \tag{2.8}$$

$$E_{\text{SNN}} = \sum_{l=1}^{L} (\text{SOPs}_l \times E_{\text{AC}}) \tag{2.9}$$

where L is the set of layers that is present in the network.

Together, the MAC–AC cost asymmetry and the input-dependent sparsity of the SOP count establish the theoretical basis for the energy efficiency claims of spiking architectures. The empirical application of this model to the trained architectures, including platform-specific extrapolation to Intel Loihi and IBM TrueNorth, is detailed in Section 3.7.6.

`3_7_sections/dynamic_power.tex` opening paragraph: strip the MAC static computation explanation and the MAC/AC energy definitions (now in Sections 2.1 and 2.6.2). Retain only the SpikeRateMonitor mechanism and the project-specific ζ derivation, with back-references to Sections 2.1 and 2.6.

`3_7_sections/energy_consumption.tex`: strip the E_{CNN}/E_{SNN} formula derivations and the Horowitz energy constants preamble (now in Section 2.6). Retain only the platform extrapolation results, the Energy Savings formula, and the results table, with back-references to Section 2.6.

2.7 Related Work

2.7.1 The Progression of Spiking Architectures

Early efforts to close the accuracy gap between SNNs and CNNs focused on classification tasks on low-resolution benchmarks. Pfeiffer and Pfeil [25] provided foundational

evidence that SNNs could exploit temporal sparsity for energy-efficient inference, while Sengupta et al. [31] demonstrated that deep SNNs could approach CNN accuracy on ImageNet via ANN-to-SNN conversion. The architectural frontier advanced further with the SEW ResNet introduced by Fang et al. [9], which resolved the vanishing gradient problem in directly-trained deep SNNs through element-wise spike residuals. Despite this depth progression, a systematic review of 151 studies by Iaboni and Abichandani [17] confirms that SNN evaluation remains predominantly anchored to low-resolution datasets such as MNIST (28×28) and CIFAR-10 (32×32); high-definition multi-object detection remains a critically underexplored frontier.

2.7.2 Event-Based Object Detection

The landscape of event-based object detection has been comprehensively surveyed by Gallego et al. [11], who established that tensor-based representations, voxel grids and event surfaces, dominate the field, typically consumed by continuous-valued detectors. The introduction of ETraM by Verma et al. [38] reinforced this pattern; despite providing the first HD event-based traffic dataset, all published baselines, RVT, RED, and YOLOv8, are continuous architectures, and no spiking detector has been evaluated against them. Furthermore, no prior work has systematically compared a spiking and a non-spiking detector on a shared HD event dataset under strict architectural parity.

2.7.3 The Gap This Project Addresses

As shown by Lin et al. [21], the thermal and power cost of deploying continuous CNNs in ADAS contexts is unsustainable. Despite this, as Pfeiffer and Pfeil [25] and the event-vision survey of Gallego et al. [11] collectively establish, the neuromorphic alternative has not been rigorously validated for high-definition multi-object detection. The `DetectorNX` project directly addresses this gap: by enforcing strict architectural parity between an SNN and a CNN baseline on the HD ETraM dataset, it provides the first controlled empirical quantification of the precision–efficiency trade-off at the spatial resolution demanded by real-world automotive perception.

Chapter 3

Technical Quality, Methodology & Evaluation

3.1 Requirements Analysis

The primary objective of this project was the development and rigorous evaluation of a high-performance Spiking Neural Network (SNN) for multi-object detection using high-definition event-camera data (the ETraM dataset). To effectively quantify the accuracy versus energy efficiency tradeoff between bio-plausible spiking architectures and traditional Convolutional Neural Networks (CNNs), the system had to be designed against a stringent set of functional and non-functional requirements. These requirements guided the iterative evolution of the neural architectures and the underlying computational pipeline.

3.1.1 Functional Requirements

To ensure a fair empirical comparison and to solve the complexities of multi-object detection on neuromorphic data, the following functional criteria were established:

- **Multi-Object Regression Capabilities:** Unlike simplified classification tasks, the system was required to perform multi-object detection capable of identifying multiple vehicles simultaneously within a single scene. This necessitated a regression head capable of outputting multiple bounding boxes per frame (defined by spatial coordinates and confidence scores) while actively suppressing "ghost detections" and resolving spatial ambiguities.

- **Architectural Parity (The "Gold Standard" Baseline):** To strictly isolate the impact of the computational paradigm (event-driven spiking versus continuous-valued activations), the baseline CNN was required to be a 1:1 non-spiking counterpart to the SNN. Both models were constrained to equivalent structures, depths and parameter counts ($\sim$ 11.7M parameters). Crucially, to eliminate data variance as a confounding factor, both the SNN and the CNN were trained on the exact same pre-processed neuromorphic dataset (the REPLICA pipeline output). This ensured that any observed discrepancies in performance or power consumption were solely attributable to the underlying network architectures, rather than differing input modalities or varied model capacity.

- **High-Definition Spatiotemporal Ingestion:** The system was required to process raw, high-definition (1280×720) event streams. Furthermore, the pipeline had to preserve the temporal dynamics of the data by segregating it into discrete time bins (e.g., $T = 10$) to allow the SNN to demonstrate temporal evidence accumulation.

- **Comprehensive Evaluation Metrics:** The evaluation suite was required to output granular metrics beyond top-line Mean IoU. This included real-time dynamic power consumption mapping, confidence calibration analysis (reliability diagrams), and quantization error tracking across varying object sizes.

3.1.2 Non-Functional Requirements & Constraints

The processing of high-definition event data at scales of up to 150 training epochs imposed severe computational bottlenecks. Consequently, the system had to satisfy critical performance and stability constraints:

- **Computational Scalability & Speed:** The SNN simulation had to be optimized for execution on local accelerator hardware (GPUs). This required the integration of Automatic Mixed Precision (AMP) to utilize Tensor Cores, memory-efficient vectorized loss functions to eliminate sequential batch loops, and the utilization of optimized C++/CUDA kernels (e.g., SpikingJelly's multi-step modes) over native Python loops.

- **Numerical Stability:** Given the reliance on 16-bit precision through AMP and the deep accumulation of membrane potentials in the SNN, the architecture required specific safeguards against gradient explosion. This constrained the design of

the detection head, mandating the use of raw logits and numerically stable loss functions (e.g., `BCEWithLogitsLoss`).

- **Hardware Extrapolation:** While constrained to local GPU execution, the benchmark outputs (e.g., dynamic power tracking) were required to be translatable into theoretical energy consumption metrics (in mJ) based on established hardware constants, facilitating projections for specialized neuromorphic chips like Intel Loihi or IBM TrueNorth.

These requirements formed the foundation for the three-generation evolution of the `SNN_MAX_REPLICA_SE` architecture detailed in Section 3.4, and drove the rigorous benchmarking suite evaluated in Section ??

Uncomment line once references are resolved

Check & Proof-read this section against rubric.

3.2 System Architecture Overview

To satisfy the stringent requirements established in Section 3.1, the overall system was designed as an end-to-end, highly modular pipeline. The modularity of this pipeline was a deliberate methodological choice to facilitate systematic design exploration. By decoupling the preprocessing, neural backbone, and regression stages, we enabled the isolation and empirical study of individual architectural modifications, ensuring that comparative findings between the spiking and continuous paradigms remained robust and controlled. The high-level data flow and structural components of the system are formalized in the block diagram presented in Figure 3.1.

As illustrated, the architecture is divided into four primary sequential stages, where the central neural backbone acts as an interchangeable module:

1. **Data Ingestion & The REPLICA Pipeline (Fixed):** Raw, asynchronous event data at a high spatial resolution of 1280×720 is ingested from the ETraM dataset. Due to the conventionally sparse and noisy nature of neuromorphic hardware outputs, the data enters the *REPLICA* preprocessing pipeline, which is heavily adapted from the methodology proposed in the original ETraM study [38]. While the core spatiotemporal noise filtering and bilinear temporal interpolation (to $T = 10$ bins) remain faithful to the original study, this project included an additional spatial downsampling stage ($ds = 4 : 1280 \times 720 \rightarrow 320 \times 180$). This

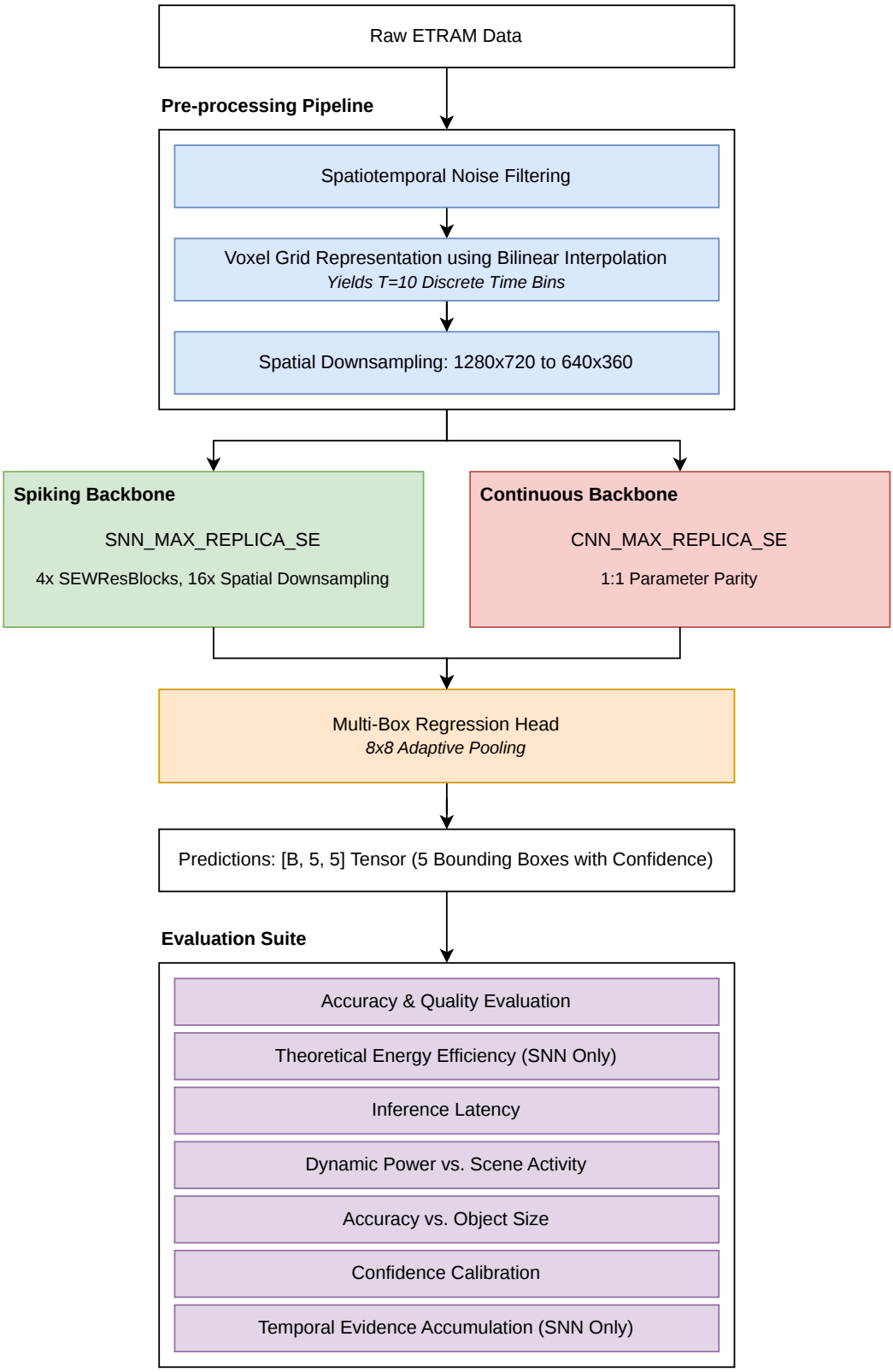


Figure 3.1: The high-level system architecture pipeline, illustrating the flow from raw ETraM high-definition event data, through the fixed REPLICA preprocessing stages, into the interchangeable neural backbone (SNN or CNN), and concluding with the multi-box regression head and the comprehensive evaluation suite.

modification was strategically implemented to ensure training feasibility on local accelerator hardware and reduce input sparsity, thereby improving the stability of neuronal feature extraction. This stage is detailed thoroughly in Section 3.3.

2. **The Neural Backbone (Interchangeable):** The preprocessed output of the *REPLICA* pipeline serves as the input to the primary feature extraction engine. This stage houses either the spiking architecture `SNN_MAX_REPLICA_SE` or its exact continuous counterpart, `CNN_MAX_REPLICA_SE`. Both backbones utilize four functional blocks incorporating Squeeze-and-Excitation (SE) mechanisms and Residual connections (`SEWResBlock`) to extract deep semantic features while performing an aggressive $16\times$ total spatial downsampling. The evolution and formal design of these backbones are detailed in Section 3.4.

3. **Multi-Box Regression Head (Fixed):** The final abstracted feature maps from either backbone are fed into a unified detection head. To resolve the spatial ambiguities and "ghost detections" observed in earlier architectural iterations, the head utilizes an 8×8 Adaptive Pooling mechanism to aggregate features from 64 spatial receptive blocks. This abstracted information is then mapped to a final output tensor of shape $[B, 5, 5]$, representing the top 5 predicted bounding boxes per image. Each detection is defined by the feature set $\mathcal{F} = \{x, y, w, h, c\}$, where (x, y, w, h) denote the normalized bounding box coordinates and c represents the object confidence logit. The regression output is evaluated via a fully vectorized Multi-Box Loss function during training.

4. **Evaluation & Benchmarking Suite (Fixed):** The predictions and internal neuronal activity from the models are passed into a specialized benchmarking suite. This stage is responsible for the rigorous empirical comparison of the two architectures across five distinct dimensions: real-time dynamic power consumption, accuracy variations relative to object size (addressing the inherent "quantization error" in SNNs [3]), model confidence calibration, temporal evidence accumulation, and latency observed on local hardware. These metrics were specifically selected to satisfy the project's success criteria and provide a comprehensive evaluation of the neuromorphic paradigm. This standardized evaluation framework ensures that the findings are scientifically robust and reproducible. This benchmarking suite is detailed thoroughly in Section 3.7.

Re-write this section, and link it to the Success Criteria from Introduction.

Using the modular pipeline guarantees that the SNN and CNN variants are evaluated identically, receiving the exact same preprocessed event volumes and outputting predictions through an identical regression and benchmarking framework.

3.3 Data Preprocessing: The REPLICA Pipeline

Raw neuromorphic event data is fundamentally different from standard frame-based video: instead of capturing absolute intensity at a fixed frame rate, an event camera asynchronously records per-pixel changes in log-intensity, outputting a continuous stream of events in the form $e = (x, y, t, p)$, where (x, y) are the spatial coordinates, t is the microsecond timestep, and $p \in -1, 1$ is the polarity of the event (indicating an increase or decrease in brightness).

While this asynchronous nature offers high temporal resolution and high dynamic range, raw event streams are inherently sparse, noisy and incompatible with conventional deep learning frameworks that expect dense, structured tensors. To bridge this gap, the raw ETraM data (1280×720 spatial resolution) was passed through a preprocessing pipeline before network ingestion.

3.3.1 Raw Event Ingestion & Windowing

The `REPLICA` pipeline begins by reading the raw h5 event files (x, y, t, p) and the partnering npy files containing the bounding box annotations from the ETraM dataset. Once both files have been parsed, the first step taken is to apply a $100ms$ sliding window across the entire duration of the event file. "Windowing" is the process of slicing the continous event stream into chunks that represent a single instance of time for the model, thereby anchoring the asynchronous event stream into discrete, structured training instances.

To maximize training efficiency and ensure a high signal-to-noise ratio in the training distribution, the `REPLICA` pipeline adopts a Box-Central sampling strategy rather than more conventional uniform temporal partitioning, where training windows were anchored deterministically around the centroids of existing ground truth annotations. This approach ensures that 100% of the training voxels contain informative structural targets, mitigating the prevalence of 'empty-scene' noise and providing the SNN with consistent temporal evidence to drive membrane potential accumulation prior to the target timestamp.

In this project, the time window $\Delta T = 100$ms was chosen to align with the temporal persistency of the human visual system of roughly 100ms [6]; just as the human eye integrates luminance changes over approximately one frame of perception, the SNN also integrates the asynchronous event stream over the same duration before producing a detection output.

The practical consequence of this choice is two-fold. First, a 100ms window is long enough to guarantee a minimum event density sufficient for the LIF neurons in the stem layer to accumulate membrane potential and produce meaningful spike trains across all $T = 10$ temporal bins. Second, specific to the context of vehicle detection, it is short enough that a vehicle travelling at highway speed (e.g., 120 km/h ≈ 33 ms^{-1}) displaces by only 3.3m within the window, ensuring that the bounding box annotation assigned to the window remains spatially valid and does not introduce label noise through excessive object displacement.

In combination with the Box-Central sampling approach, the pipeline applies a density filter prior to voxelization to enforce a minimum quality threshold in valid samples. For each 100ms window, the total number of raw (x, y, t, p) events is counted: windows containing fewer than `min_events` $= 500$ events are discarded, while those meeting or exceeding this threshold proceed to voxelization as valid training samples. This filter is a structural necessity for SNN training stability:

- **Prevention of dead neurons:** SNNs are metabolic models: they require energy (spikes) to learn. Training on near-empty windows rewards the optimiser for suppressing all firing, driving synaptic weights so low that even high-density frames containing real vehicles fail to trigger a spike response [8].

- **Firing Rate Regularization (FRR) stability:** The FRR loss penalises deviation from a target firing rate of 20% [5, 39]. Presenting near-empty inputs forces the regularisation term to dominate the total loss, as it attempts to generate spikes in the absence of any meaningful input signal.

- **Gradient quality:** A forward pass producing no spikes yields a zero surrogate gradient [23]. Training on empty windows is therefore mathematically degenerate since no weight update occurs, so High Performance Computing (HPC) cycles are consumed without any learning signal.

- **Background activity suppression:** Event cameras produce a baseline level of Background Activity (BA) noise from transistor mismatch and thermal effects

[11]. A window containing less than 500 events is statistically likely to consist almost entirely of noise; the `min_events` threshold therefore functions as a signal intensity floor, directing the model's attention exclusively towards windows in which a meaningful scene change has occurred.

3.3.2 Signal Conditioning

THe first step of the `REPLICA` pipeline is to apply a series of signal conditioning operations to the raw event data to improve the Signal-to-Noise Ratio (SNR), which yields several benefits:

- **Prevention of LIF neuron saturation:**

  Without input conditioning, high-magnitude or noisy voxel values can drive every neuron in the initial stem layer to fire continuously and flood the network with uninformative spikes. By normalising the voxel tensor to the range $[0, 1]$, the input magnitude is constrained such that only neurons encoding genuine structural features,such as vehicle edges and motion boundaries, exceed their firing threshold, preserving the sparsity and selectivity of the deeper backbone stages [8].

- **Improved objectness discrimination:** Background artifacts inherent to HD event cameras, such as flickering illumination and sensor noise, generate erraneous event clusters that appear similar to small objects. By rejecting these low-activity events during the Background Activity Filtering (BAF) denoising step, the network is not required to expend representational capacity on learning to suppress noise, and can instead focus on learning the geometric structure of real vehicles [8, 29].

- **Reduced synaptic operations and neuromorphic energy efficiency:** Since SNNs consume energy only upon spike emission, every erraneous background event that is suppressed during preprocessing directly reduces the number of SOPs at inference time. This establishes a direct link between preprocessing quality and the power efficiency of the architecture when deployed on neuromorphic hardware [29].

The Signal Conditioning segment of the `REPLICA` pipeline was lifted directly from the original ETraM study, since it was designed specifically to suit the characteristics of the dataset [38].

3.3.3 Spatiotemporal Voxelization

To make the asynchronous event data compatible with neural network layers, a Voxelization process is performed to transform the continuous stream into a structured tensor. This process consists of three core operations:

- **Temporal Binning:** The total duration of an event sample (e.g., a 100ms window) is divided into 10 discrete time bins ($T = 10$).
- **Polarity Splitting:** Events are separated into two distinct channels based on their polarity ($C = 2$), representing positive and negative brightness changes.
- **Event Accumulation (Histogramming):** Within each discrete temporal bin and polarity channel, the individual asynchronous events are aggregated at their respective spatial coordinates (x, y). Specifically, the value of each "pixel" in the resulting spatial grid becomes the total count of events (of that specific polarity) that occurred at that exact location during that fraction of time. This process effectively converts the sparse, continuous point-cloud of events into a sequence of dense 2D frames—or event histograms—which standard convolutional and spiking layers can natively process.

This accumulation process preserves the temporal distribution of the events while generating a dense 4D spatiotemporal tensor of shape $[10, 2, 720, 1280]$ (representing Time $\times$ Channels $\times$ Height $\times$ Width).

3.3.4 Downsampling & Normalization

Following spatiotemporal voxelization, the voxelized data undergoes a dual-phase refinement consisting of spatial downsampling and amplitude normalization. This is the final polishing step of the pipeline, transforming the high-resolution event counts into a stabilized, low-memory format that the neural backbones can process efficiently.

Spatial Downsampling (16-fold Data Reduction)

The generated high-definition voxel grid (1280×720) is subjected to a 4×4 Max-Pooling operation, reducing the spatial resolution to 320×180. This 16-fold reduction in data volume is critical for managing the high-dimensional state-space of the SNN backbone. Because SNNs must store neuron membrane potentials across time, an HD model would require massive amounts of GPU memory. By aggressively reducing

the spatial dimensions, we ensure the backbone scalability, making it computationally feasible to train the deep 4-block architecture over extended epochs.

Furthermore, utilizing Max-Pooling (as opposed to Average-Pooling) acts as a feature preserver: it retains the strongest event signal within each 4×4 area, helping the network maintain the crisp, high-contrast edges of the vehicles even at a lower resolution.

It should be noted that the downsampling factor $ds = 4$ was heavily influenced by the HPC cluster's GPU memory constraints; without using $ds = 4$, the data would not fit into the GPU memory, or would require a batch size of 1, making training infeasible within the time constraints of this project.

Max-Normalization (Input Calibration)

Subsequently, max-normalization is performed to scale the raw event accumulation counts into a continuous $[0, 1]$ interval, by dividing every pixel in the temporal volume by the single highest event count found within that 10-bin voxel.

This normalization serves as a fundamental safeguard against neural saturation. LIF nodes have fixed firing thresholds (in this architecture, $V_{th} = 0.2$) - if a raw, unscaled event count of 50 were fed into the network, the first-layer neurons would fire instantaneously and repeatedly, effectively "screaming" and drowning out all other spatiotemporal information. By enforcing input calibration, we ensure that the input magnitudes are calibrated relative to the LIF thresholds, facilitating stable surrogate gradient propagation during the 150-epoch training cycle.

3.3.5 Multi-Object Label Alignment

The final stage of the pipeline implements a Multi-Object Label Alignment strategy, which is fundamental to the architecture's multi-object detection capabilities. This stage transforms individual, asynchronous bounding box annotations into structured, scale-independent target tensors suitable for the network's multi-box regression head.

Timestamp-Based Grouping (Unified Scene Representation)

Instead of creating isolated training samples for every individual vehicle, the pipeline parses the ground truth annotations and groups all vehicles that exist at the exact same microsecond (t). This process merges individual detections into a Unified Scene Representation, which is crucial because it eliminates "label noise"; presenting the

network with identical visual input but different single-car targets across multiple samples would force the model to average its predictions, resulting in blurry, low-confidence bounding boxes with poor IoU scores.

Fixed-Capacity Padding

Deep learning models inherently require consistent tensor dimensions for batched training. To accommodate scenes with varying numbers of vehicles, we enforce a Fixed-Capacity Interface of exactly 5 bounding boxes per frame. If a scene contains 2 vehicles, the remaining 3 slots are populated with "empty" zero-padded boxes. If a frame contains no vehicles, 5 empty boxes are provided. This ensures every target label is a consistent $[5,5]$ tensor. The limit of 5 boxes was empirically selected based on an analysis of the ETraM validation set, where the vast majority of frames contain 3 or fewer vehicles, making 5 a safe upper bound for this specific traffic monitoring environment.

Binary Masking

To enable the network to distinguish between genuine vehicle targets and zero-padded instances, the tensor utilizes a Binary Masking dimension. For every box, the target tensor stores five values: $[Center_X, Center_Y, Width, Height, Mask]$.

- For a real object, the mask value is set to 1.0.
- For a padded or empty box, the mask value is set to 0.0.

This mask acts as the objectness confidence target. It is utilized by the Multi-Box Loss function to selectively apply coordinate gradients only to valid targets, while simultaneously regularizing the model's confidence head to output 0.0 for empty regions.

Normalization to Sensor Space

Finally, all spatial parameters (center coordinates, width, and height) are originally provided in raw pixel values (e.g., $x = 842$). These are normalized by dividing by the raw sensor width (1280) and height (720), scaling all coordinates into the continuous range $[0.0, 1.0]$. This provides a Scale-Independent Signal, ensuring the regression head can learn the underlying geometry robustly, regardless of the spatial downsampling factors applied earlier in the pipeline.

3.3.6 Chunking for High-Throughput I/O

To optimize throughput on the HPC cluster, the precomputed dataset was encapsulated into a High-Throughput Chunking architecture. Without this optimization, a 150-epoch training run on the high-definition event data would likely take days due to severe I/O bottlenecks.

The Chunking Mechanism

Rather than saving 10,000 individual files representing every single frame, the pipeline bundles the data into large, contiguous blocks, or "chunks" (e.g., 250 or 500 samples per file), where the chunk size is defined by the memory constraints of the environment used for training. Each chunk is saved as a single, compressed PyTorch (`.pt`) file containing two massive tensors: the spatiotemporal voxels and their corresponding aligned targets. To manage this structure, a global `manifest.json` file acts as a map, recording exactly which sample index resides in which chunk and specifying its byte-offset.

Mitigating HPC Bottlenecks

This chunking strategy was an absolute necessity for efficient execution on the cluster, addressing three primary hardware constraints:

- **Solving the "Small File Problem" (Metadata Latency):** HPC systems typically utilize a Network Attached Storage (NAS/NFS) architecture. Opening a file on a network drive incurs high *Metadata Latency*. If the GPU were forced to open 10,000 independent files per epoch, it would spend the vast majority of its time idling while waiting for the network to locate the next file. By encapsulating the data into chunks, the HPC only needs to open a fraction of the files (e.g., 40 files for 10,000 samples). This allows the system to perform massive *Sequential I/O* reads, which are orders of magnitude faster than random access reads on networked drives.
- **Bypassing CPU Precomputation Bottlenecks:** In deep learning pipelines, the CPU often acts as the data loader for the GPU. Preprocessing 1280×720 event data into voxel grids on-the-fly is highly computationally expensive.; by enforcing strict *Precomputation* and chunking everything prior to training, the training script can simply copy the structured tensors from disk directly into VRAM.

This ensures the CPU is no longer a bottleneck, maintaining *Maximum GPU Utilization* and enabling iteration speeds as low as 160ms/it.

- **Efficient Memory Caching (LRU):** The chunked architecture enabled the implementation of a custom `VoxelDiskDataset` featuring a Least Recently Used (LRU) memory cache. Because the data is bundled, the HPC can load entire blocks of 250 samples directly into its extensive RAM (e.g., 120GB). Consequently, during subsequent epochs, the model frequently bypasses disk I/O entirely, pulling chunks directly from RAM and making data loading nearly instantaneous.

By utilizing chunk-based sequential I/O and a memory-mapped caching layer, the *REPLICA* pipeline successfully bypassed the CPU-loading bottleneck, ensuring maximum GPU utilization and reducing the total 150-epoch training time from a projected several days to approximately six/ seven hours.

3.3.7 REPLICA Pipeline: Visualized

Figure 3.2 details the order of which the aformentioned preprocessing steps are applied to the raw event data.

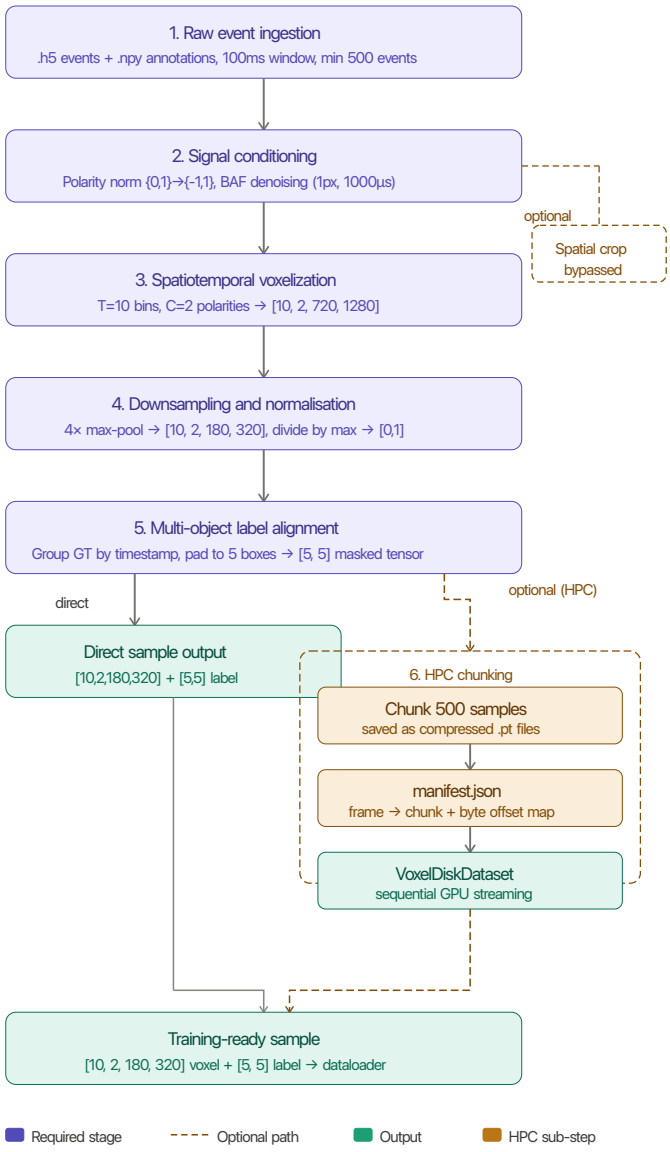


Figure 3.2: Overview of the spatiotemporal preprocessing pipeline applied to the ETraM dataset. Raw events are voxelized into a 4D tensor, spatially downsampled, and aligned with multi-object ground truth labels to produce a final sample of shape `[10, 2, 180, 320]` with a corresponding label tensor of shape `[5, 5]`.

3.4 Spiking Neural Network (SNN) Architecture

The core focus, effort and innovation of this project is the development and optimization of the `SNN_MAX_REPLICA_SE` architecture. This model is explicitly designed to process high-definition neuromorphic event data while adhering to the computational paradigms of SNNs. By prioritizing event-driven, binary communication, the architecture aims to drastically reduce both the dynamic power consumption and the computational complexity compared to traditional deep learning models, making it superior for power and time constrained environments.

3.4.1 Spatiotemporal Input & Multi-Step Execution

The raw ETraM event data is initially preprocessed to a spatial resolution of 640×360 with $T = 10$ discrete temporal bins. Consequently, the network ingests a massive, high-dimensional spatiotemporal tensor of shape $[B, 10, 2, 360, 640]$ (Batch, Time, Polarity, Height, Width).

Given the high spatial dimensionality and the iterative nature of spiking neurons, native Python loop execution over the time dimension proved computationally restrictive. To overcome this bottleneck, the entire architecture was refactored to utilize the SpikingJelly framework's optimized Multi-Step (`step_mode='m'`) execution paradigm [10]; instead of iterating through the 10 time bins sequentially, the entire spatiotemporal tensor is passed directly into custom-compiled Compute Unified Device Architecture (CUDA) kernels. This low-level optimization enables high-speed, parallelized execution of the temporal dynamics, making the extensive benchmarking of this HD dataset feasible on local GPU hardware, and reducing overall training time from approximately 16 hours to just 7.

3.4.2 Neuron Dynamics: LIF

At the foundation of the architecture lies the LIF neuron model, which governs the temporal dynamics and binary spike generation of the network. Unlike traditional ANNs that utilize continuous activation functions such as ReLU, the LIF node maintains an internal state: the membrane potential $V[t]$, which updates iteratively across discrete time steps $t \in [1, T]$.

The Forward Pass (Spike Generation)

The dynamics of the LIF neuron at a given spatial position and time step t are governed by the integration of incoming pre-synaptic spatial features $X[t]$ and an inherent leak factor τ. The hidden membrane potential $H[t]$ prior to spiking is mathematically defined as:

$$H[t] = V[t-1] + \frac{1}{\tau}(X[t] - (V[t-1] - V_{reset})) \tag{3.1}$$

When $H[t]$ exceeds a predefined threshold V_{th}, the neuron emits a binary spike $S[t] \in \{0,1\}$ to the subsequent layer, governed by the Heaviside step function $\Theta(x)$:

$$S[t] = \Theta(H[t] - V_{th}) \tag{3.2}$$

Following a spike ($S[t] = 1$), the neuron undergoes a hard reset, instantly dropping its potential $V[t]$ to V_{reset}. If no spike is emitted ($S[t] = 0$), the potential naturally decays, ensuring that subsequent layers only perform computations (Synaptic Operations) upon receiving active events [31]. This "activation sparsity" is the primary driver of the SNNs exceptional energy efficiency [28].

The Backward Pass (Surrogate Gradients)

Training deep SNNs via backpropagation presents a fundamental mathematical challenge, since the Heaviside step function governing spike generation (Equation 2) is non-differentiable. Its derivative is zero everywhere except at the threshold V_{th}, where it is infinite. Using conventional backpropagation here would immediately result in zeroed gradients, halting the learning process - this is a phenomenon known as the "dead neuron" problem.

To resolve this, the network employs a Surrogate Gradient during the backward pass [24]. While the forward pass remains strictly binary, the backward pass approximates the step function using a smooth, differentiable curve. This specific architecture uses the ATan function:

$$\frac{\partial S}{\partial H} \approx \frac{\alpha}{2\left(1 + \left(\frac{\pi}{2}\alpha(H - V_{th})\right)^2\right)} \tag{3.3}$$

Where α is a hyperparameter controlling the steepness of the surrogate curve. This mathematical substitution allows the network to learn complex spatiotemporal representations from scratch while maintaining a purely binary forward execution.

3.4.3 Backbone: SEW-ResNet & Attention

To extract high-level semantic features and manage the computational complexity of the high-definition input, the architecture utilizes a deep residual backbone divided into five sequential stages, as outlined in Table 3.1.

Stage	Name	Resolution	Channels	Primary Operation
1	Initial Stem	90×160	64	Stride-2 Conv + LIF (Downsamples HD input)
2	Backbone Alpha	22×40	256	Blocks 1 & 2 (SEWResBlocks with Stride-2)
3	Attention Bridge	22×40	256	Squeeze-and-Excitation (SE) Global Firing Recalibration
4	High-Res Backbone	22×40	512	Blocks 3 & 4 (SEWResBlocks with Stride-1)
5	Temporal Fusion	22×40	512	Summation of spikes across all T=10 timesteps

Table 3.1: Detailed breakdown of the five sequential stages in the `SNN_MAX_REPLICA_SE` backbone.

Spatial Downsampling

As illustrated, the architecture implements an aggressive $16\times$ total spatial downsampling strategy to compress the initial 640×360 input. This is achieved through strided convolutions applied within the Initial Stem and Backbone Alpha, progressively reducing the spatial dimensions to a highly compressed 22×40 grid while expanding the channel width up to 512.

Spike-Element-Wise (SEW) Residual Blocks

Building deep SNNs presents significant challenges due to the "vanishing spike" problem inherent in standard residual connections. In a standard ResNet, the residual addition occurs *before* the activation function. However, if we follow this standard and add two binary spike trains, the result is a non-binary, continuous value (e.g., $1 + 1 = 2$), which destroys the energy-efficient, event-driven nature of the network [9].

To overcome this, the `SNN_MAX_REPLICA_SE` architecture adopts the SEW residual paradigm [9], instantiated as `SEWResBlock` units throughout the backbone. Unlike standard Spiking ResNets, SEW ResBlocks apply the element-wise residual operation *after* the spiking activation function, guaranteeing that the output remains strictly binary. Fang et al. [9] prove that this reordering is both necessary and sufficient to achieve

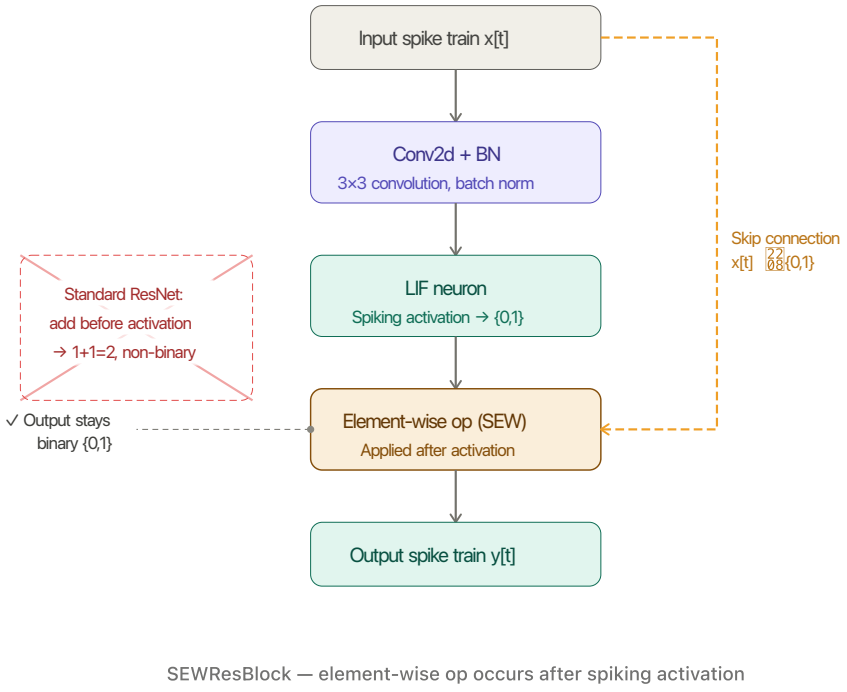


Figure 3.3: The SEW residual block (`SEWResBlock`) as proposed by [9].

identity mapping in a purely spike-based network, resolving the degradation problem without sacrificing neuromorphic compatibility. In this architecture, this property is exploited to stack 4 SEWResBlocks across the backbone stages (Stages 2 and 4 in 3.1), enabling representational depth that would otherwise cause spike degradation under a standard residual formulation.

Structural Attention: SE-Block

In deep learning, a bottleneck refers to a point in the neural network where the information flow is forced through its most compressed and abstract representation. In this `SNN_MAX_REPLICA_SE` architecture, the bottleneck exists at the transition point between stages 2 and 3 of the backbone, and represents two things:

1. **A Spatial Constraint (Resolution Limit):** At this stage, the original 180×320 spatiotemporal input has been downsampled by a factor of 8x via the strided convolutions in the stem and early residual block, resulting in a highly compressed22×40 feature map. This spatial bottleneck is critical because each remaining ”pixel” must now represent a large 8×8 receptive field of the original HD sensor data,

forcing the network to discard fine-grained geometric redundancy while retaining the core structural boundaries of multiple detected objects.

2. **A Semantic Shift (High-Level Abstraction):** This juncture marks the definitive pivot where the network transitions from extracting low-level geometric primitives, such as spike-edge orientation and motion direction, to synthesizing complex semantic concepts. Without an explicit selection mechanism, the network at this depth is susceptible to noise and artifacts from the HD event stream, which can blur the distinction between legitimate targets and background interference.

To both mitigate and solve these bottleneck constraints, a Squeeze-and-Excitation (SE) mechanism is introduced in Stage 3 of the backbone, sitting directly at the transition point [16]. The SE block operates by performing a global average pooling of the 256 feature channels (the "Squeeze" phase), followed by two fully connected layers that calculate a weighting factor for each channel (the "Excitation" phase). This effectively mitigates the spatial constraint by allowing the network to incorporate global context into its local representations, compensating for the loss of resolution.

When applied in the context of an SNN, this acts as a dynamic firing modulator: the backbone's activations are re-calibrated by looking at the global context of the entire frame, magnifying the most informative channels whilst suppressing those containing artifacts. By applying this recalibration, the SE block ensures that only the most relevant "Objectness" (the probability that a specific region of an image contains a generic object of interest rather than background noise) features are passed into the final high-capacity blocks, effectively mitigating the semantic ambiguity before the final bounding box regression.

3.4.4 Firing Rate Regularization (FRR)

A common failure mode in deep SNN training is network silencing (zero spikes emitted) or over-saturation (firing continuously at every timestep), both of which halt gradient flow and destroy energy efficiency. To constrain the network, a custom Firing Rate Regularization (FRR) Metabolic Penalty was integrated into the training pipeline.

The FRR applies a squared error loss to the network's average firing rate:

$$L_{frr} = \lambda \cdot (Rate_{avg} - Target)^2 \tag{3.4}$$

This biologically inspired metabolic constraint [29] forces the 11.7M parameter

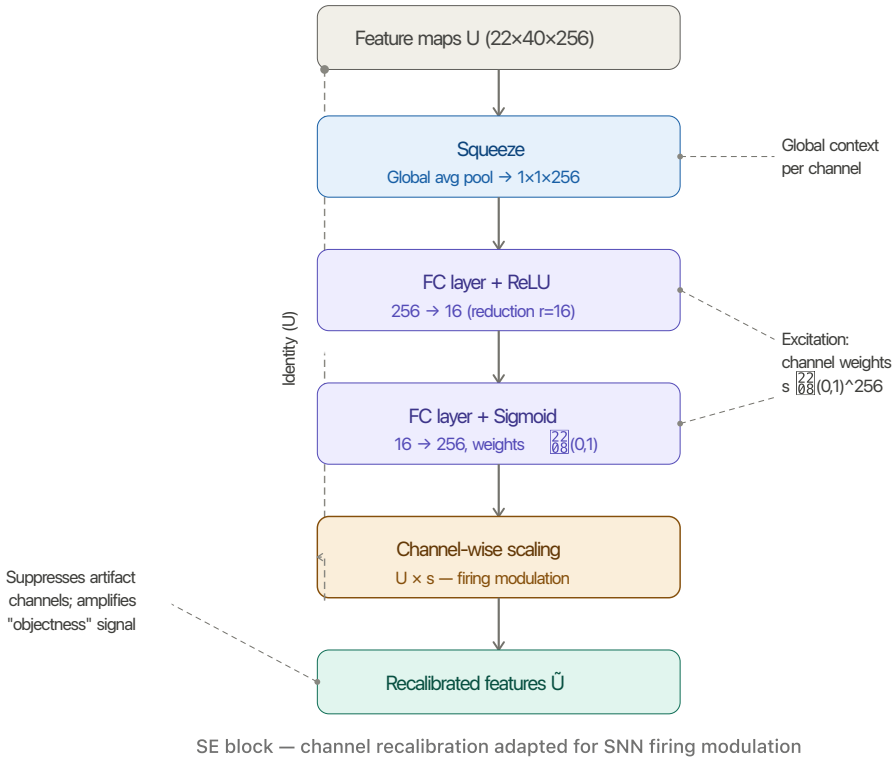


Figure 3.4: The Squeeze-and-Excitation (SE) block [16] as applied in Stage 3 of the `SNN_MAX_REPLICA_SE` backbone ($22\times40\times256$).

backbone to maintain a selective but informative firing rate (e.g., $Target = 20\%$) across its hidden layers. Penalising derivation from a target firing rate is an established technique for preserving activation sparsity in directly trained SNNs [5, 39]. Adding this penalty to the model's loss ensures high information flow while strictly preserving the activation sparsity required for efficient neuromorphic deployment.

3.4.5 Multi-Box Regression Head

Following the final backbone stage, the abstract spatiotemporal feature maps are fed into a unified detection head. To process the spatial information effectively and mitigate spatial ambiguities observed during the iterative design process (detailed in Section 3.6), the head utilizes an 8×8 Adaptive Average Pooling mechanism.

This pooling operation aggregates the features into 64 distinct spatial receptive blocks. The pooled features are then mapped via final convolutional and linear layers to an output tensor of shape $[B, 5, 5]$, where B represents the batch size. The final

two dimensions encode the top 5 predicted bounding boxes globally per image, with each box defined by a feature tuple $\mathcal{F} = \{x, y, w, h, c\}$. Here, (x, y, w, h) represent the normalized spatial coordinates and dimensions of the bounding box, and c denotes the object confidence logit. This formulation allows the network to predict multiple vehicles simultaneously, which are subsequently evaluated using a vectorized Multi-Box Loss function.

Add model architecture diagram.

Check & Add model hyperparameter table.

3.5 Convolutional Neural Network (CNN) Baseline

The CNN architecture `CNN_MAX_REPLICA_SE` was engineered as the primary baseline to evaluate the SNN against. It is a 11.73M parameter CNN specifically designed to mirror the SNN architecture, satisfying the "Architectural Parity" functional requirement specified in 3.1.1. This ensures that the only variable being tested in this comparative study is the neural activation mechanism (event-driven binary spikes versus continuous-valued floats), making the comparison immune to common criticisms regarding unmatched or unoptimized baselines.

3.5.1 Structural Configuration & Layer-Wise Mapping

The CNN mirrors the SNN's configuration strictly 1:1, replacing every temporal and spiking component with a standard continuous equivalent. Table 3.2 outlines this direct mapping.

3.5.2 Input Strategy: Channel-Wise Temporal Integration

Unlike the SNN, which processes event data sequentially over discrete time steps T, the CNN baseline utilizes a flattened temporal input strategy. Instead of processing 10 timesteps sequentially, it concatenates the $T = 10$ bins (each containing 2 polarity channels) into a single input tensor of 20 channels, changing the input shape from $[B, 10, 2, H, W]$ to $[B, 20, H, W]$.

This approach allows the CNN to correlate events across the entire 100ms window in a single spatial pass. This channel-wise stacking of event bins is a standard and effective methodology for feeding asynchronous event-camera data into traditional, non-spiking CNN architectures [12].

SNN Component	CNN Equivalent	Technical Purpose
Spatiotemporal Voxel `[B, 10, 2, H, W]`	Flattened Channel Input `[B, 20, H, W]`	Translates 10 time bins into a format the respective network can ingest.
LIF Neuron (Leaky Integrate-and-Fire)	ReLU Activation	Provides the nonlinearity; SNN uses discrete pulses, CNN uses continuous values.
Binary Spikes $\{0,1\}$	Continuous Floats $[0,\infty)$	Represents the information "currency" being passed between layers.
Surrogate Gradient	Exact Derivative (Standard)	Enables backpropagation through the non-differentiable spiking threshold.
SEWResBlock (Spiking Shortcut)	Standard Residual Block	Implements identity mapping to prevent vanishing gradients during training.
Multi-Step Mode (`step_mode='m'`)	Single spatial pass	Processes the temporal window efficiently on the GPU.
Squeeze-and-Excitation (Spike-based)	Squeeze-and-Excitation (Float-based)	Recalibrates channel importance based on global frame context.
Temporal Summation ($\sum_{t=1}^{T} S_t$)	N/A (Direct Feature Pass)	Aggregates 100ms of evidence into a single map for the regression head.
Multi-Box Head (5 Boxes)	Multi-Box Head (5 Boxes)	Predicts coordinates and objectness scores for 5 targets simultaneously.
Synaptic Operations (SOPs)	Floating Point Ops (FLOPs)	The metric used to quantify and compare hardware energy efficiency.

Table 3.2: 1:1 Architectural Mapping (SNN vs. CNN)

3.5.3 Backbone: Identity Mapping & Residual Connections

The CNN backbone employs deep residual learning, utilizing four Residual Blocks that expand to 512 channels in the final stage. To satisfy the functional requirement of Architectural Parity, the spiking `SEWResBlock`s from the SNN were replaced with Standard Residual Blocks. The standard structure within these blocks is Conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Conv $\rightarrow$ BatchNorm + Shortcut $\rightarrow$ ReLU.

The inclusion of these skip connections (shortcuts) is vital for enabling the CNN network to avoid the same vanishing gradient problem seen in its SNN counterpart, and successfully learn complex vehicle geometries across the 320×180 field of view [14].

3.5.4 SEWResBlock vs Standard ResBlock

The core mathematical difference between the two architectures lies in their summation logic and the resulting information flow across the shortcut connection:

- **Standard ResBlock (CNN):** Performs an accumulative floating-point addition ($y = f(x) + x$). The continuous intensity of the residual path is directly added to the identity path before the final ReLU activation. Here, the shortcut carries the *raw input features* forward. This mechanism allows the model to learn the "difference" (the residual) needed to improve the representation, focusing on information preservation [14].

- **SEWResBlock (SNN):** Performs a relational element-wise spiking operation *after* the activation function. Because adding two binary spikes (e.g., $1 + 1 = 2$)

produces a non-binary value, the SNN uses a spike-merging function (such as ADD or AND) to preserve the discrete $\{0,1\}$ state. Crucially, the SNN shortcut carries the *membrane history* forward, prioritizing temporal integrity.

Table 3.3 summarizes these foundational differences in identity mapping.

Feature	**Standard ResBlock (CNN)**	**SEWResBlock (SNN)**
Summation	Accumulative ($y = f(x) + x$)	Relational (Spiking Element-Wise)
Output Type	Continuous Float $[0, \infty)$	Binary Spikes $\{0,1\}$
Primary Benefit	Prevents vanishing gradients	Prevents spike vanishing
Mathematical Goal	Information preservation	Temporal integrity

Table 3.3: Comparison of identity mapping between Standard ResBlocks and SEWResBlocks.

This distinction is particularly relevant for high-speed neuromorphic data: while the Standard ResBlock accumulates continuous values that effectively blur fast movements into a single averaged float over the 100ms window, the SNN equivalent preserves exact spike timing, maintaining the temporal precision required for the regression head.

3.5.5 Attention Mechanism: SE-Block

The CNN network encounters the same informational bottlenecks as its SNN counterpart, detailed in Section 3.4.3. Consequently, it implements the same architectural solution via an SE Block [16]. However, the `CNN_MAX_REPLICA_SE` implements this mechanism in the continuous domain: as opposed to utilizing spike rates, the CNN employs a float-based approach.

During the 'Squeeze' segment, the CNN utilizes global average pooling to calculate the mean spatial intensity of features per channel, rather than calculating a temporal firing rate. Subsequently, during the 'Excitation' phase, the block outputs a learned coefficient that scales the input activations. This effectively recalibrates the channel-wise feature responses, as opposed to modulating discrete membrane potentials.

While the integration of an SE Block in a standard CNN improves feature representation, it does not confer the same critical hardware-level energy sparsity benefits as it does within an SNN. The primary advantage of the SE mechanism in this specific context is the enforcement of structural sparsity in the spiking domain. Nevertheless, to strictly adhere to the Architectural Parity requirement established in Section 3.1.1, the continuous equivalent of this mechanism was preserved in the baseline.

3.5.6 The "Gold Standard" Upper Bound

By constructing this custom CNN mirror, the baseline serves as the upper theoretical bound of this specific architecture's capability on the ETraM dataset. For example, if the CNN baseline achieves a Mean IoU of 48%, this represents the maximum performance extractable by this parameter configuration. If the corresponding SNN achieves 43% IoU, the 5% gap explicitly represents the "quantization cost" of switching from 32-bit continuous floats to 1-bit binary spikes [3]. Adhering to this methodology ensures that the evaluation of the neuromorphic paradigm is both fair and scientifically sound.

Add model architecture diagram.

Check & Add model hyperparameter table.

3.6 Architectural Evolution

The final **ULTRA** revision of the `MAX_REPLICA_SE` architecture was not the result of a single design phase, but rather the culmination of an iterative engineering process across four distinct generations. According to the project's success criteria, the model required both high temporal precision and robust spatial localization. This section documents the empirical obstacles encountered during development, and the actionable insights that led to the final optimized configuration.

3.6.1 Phase 0: The Spatial Cropping Pitfall (Early Prototyping)

Prior to the implementation of downsampling in the `REPLICA` pipeline, early prototyping explored the use of aggressive spatial cropping to bypass the computational costs of high-definition event processing.

Architectural Intent & Technical Logic

The primary objective of this phase was to establish a high-resolution localized baseline by cropping the raw 1280×720 ETraM sensor data into a 224×224 patch, as illustrated in Figure 3.2; these patches were deterministically anchored to the centroid of the ground-truth vehicle annotation.

The technical logic behind this approach was rooted in improving the SNR at the network's input. By utilizing *localized foveation*, we hypothesized that the initial stem layers would receive a significantly higher frequency of spikes per pixel for the

target object. This increased spike density was intended to facilitate the extraction of fine-grained geometric features which are often attenuated or lost during global spatial downsampling, whilst also making it easier for the network to learn the features of vehicles by enlarging and zooming in on them. Additionally, since the localized patches occupied a smaller memory footprint, they could be processed without the need for aggressive pooling, and preserve the original sensor resolution.

Empirical Obstacle: Constant Target Bias

During initial training cycles, the model achieved near-instantaneous convergence and exceptional accuracy metrics. However, a critical failure in the methodology was identified during validation: the model had failed to learn any meaningful feature representation.

Because the pipeline utilized deterministic centroid-anchored cropping, the target vehicle occupied the geometric center of every training sample. Consequently, the SNN succumbed to *Constant Target Bias*—a severe form of spatial overfitting where the regression head ignores the visual event stream and instead learns to output a constant coordinate vector of approximately $[0.5, 0.5, w, h]$. This eliminated the localization signal entirely; the model stopped "searching" for the vehicle and simply memorized the center coordinates. Such a model is fundamentally useless for real-world traffic monitoring where vehicle positions are stochastically distributed across the full field of view, and are not pre-known to use spatial cropping before inference..

Insert images of the constant target bias.

Actionable Insight

This phase provided a critical methodological lesson: for a spiking architecture to learn robust spatial localization and objectness, it must be exposed to targets moving dynamically relative to the sensor boundaries within a full-frame (or semi-full-frame) field of view. This insight led to the total abandonment of centered cropping in favor of the global $4\times$ spatial downsampling strategy utilized in the final *REPLICA* pipeline, ensuring that the model learned to correlate visual spike patterns with absolute spatial coordinates.

3.6.2 Generation 1: The Multi-Box Paradigm Shift

Following the resolution of the spatial cropping bias, the architecture progressed to full-frame processing. However, this transition immediately exposed fundamental limitations in the network's predictive capabilities regarding multi-object scenes.

Architectural Intent & Technical Logic

The primary objective of this phase was to develop a *scene-aware detector* capable of identifying multiple vehicles simultaneously within the ETraM traffic environment. To achieve this, the network's output topology underwent a paradigm shift, transitioning from a simplistic single-box regression head (which output a single $[x, y, w, h]$ vector) to a Multi-Box Regression Head with a fixed-capacity output of $[B, 5, 5]$, where there are 5 features $(x, y, w, h, mask)$ and 5 bounding boxes predicted per inference. The technical logic dictated that the network required a mechanism to independently evaluate distinct spatial regions. To facilitate this, each of the five predicted boxes was augmented with a fifth dimension: an "Objectness" or Confidence Score. This structural addition allowed the model to not only regress bounding box coordinates but also to gate its own predictions, learning to suppress background artifacts with low confidence while maintaining high confidence for legitimate structural targets.

Empirical Obstacle: Single-Target Ambiguity & Label Noise

Initial prototypes utilizing the standard single-box regression head proved fundamentally incompatible with real-world traffic scenes. When multiple vehicles occupied the field of view simultaneously, the model faced a logical contradiction.

For instance, if a scene contained both Car A and Car B, the SNN backbone would naturally extract features for both. However, if the training label only provided the coordinates for Car A, the loss function would heavily penalize the network's correct detection of Car B as an error, introducing severe label noise. Over thousands of iterations, the model attempted to minimize this ambiguous loss by mathematically averaging the positions of all visible vehicles. This catastrophic failure mode resulted in a predicted bounding box that hovered uselessly in the empty space between vehicles, or a singular "super-box" attempting to encompass the entire roadway, leading to a collapse in IoU performance.

Insert images of the super box.

Actionable Insight: The Vectorized Matching Loss

This phase highlighted that resolving spatial ambiguity required a sophisticated mechanism to handle the *Assignment Problem*—the mathematical challenge of determining which of the 5 predicted boxes should be compared against which ground truth vehicle.

The actionable insight was the development of a Vectorized Multi-Box Loss. This computational innovation calculates a 3D matrix of all possible IoU overlaps between the 5 predictions and the M ground truth objects in every batch. Utilizing greedy matching, the loss function identifies the best match for each real vehicle in the scene. Only this matched predicted box receives a *coordinate gradient*, signaling it to move closer to the target.

Crucially, all unmatched predicted boxes are assigned a target confidence of 0.0, acting as a metabolic penalty for false positives. This forces the model to learn that predicting a vehicle where none exists carries a heavy mathematical penalty, effectively cleaning up the SNN's vision and reducing the Ghost Detections prevalent in early iterations.

By transitioning to a multi-box architecture, we moved the SNN from a simplistic regression model to a sophisticated scene-aware detector, and introduced the Multi Box Label Alignment step shown in figure 3.2 into the `REPLICA` pre-processing pipeline. This shift allowed the network to resolve the spatial ambiguity of multi-vehicle traffic, transforming the contradictory Label Noise of previous versions into a structured multi-target learning signal. This evolution was essential for achieving stable convergence on the high-definition, high-density event streams of the ETraM dataset.

3.6.3 Generation 2: The Expanded Spatial Head (7.00M Parameter `MAX` Generation)

While Generation 1 successfully established the multi-box logic, empirical validation revealed a persistent failure mode characterized by ghost Detections and overlapping predictions.

Architectural Intent & Empirical Obstacle

The primary objective of this phase was to resolve *Information Congestion* within the regression head. Generation 1 architecture utilized an Adaptive Average Pooling layer of 4×4, compressing the entire field of view into just 16 spatial blocks before entering the final linear layers.

This severe spatial bottleneck meant that when vehicles were in close proximity, or situated near clusters of sensor artifacts, their respective spiking features were averaged into an identical 4×4 block. The regression head lacked the spatial memory required to distinguish valid target spikes from background noise. Consequently, the model frequently outputted overlapping bounding boxes for a single vehicle or failed to suppress high-confidence noise, resulting in an inflated average of 2.50 detections per frame - significantly higher than the ground truth average of 2.0.

Actionable Insight: Quadrupling Spatial Memory

To resolve this congestion, the Adaptive Pooling resolution was expanded from 4×4 to an 8×8 grid, quadrupling the available spatial memory to 64 distinct blocks, providing the linear regression head with a substantially finer structural map of the backbone's activations.

This architectural refinement enabled localized Object-Noise Discrimination: with 64 distinct blocks, the model could successfully isolate the structured spike trains of a vehicle from the unstructured noise of adjacent sensor artifacts. This resulted in dramatic Confidence Polarization: max confidence scores for real objects surged to 0.99, while background noise was successfully pushed toward 0.0.

Consequently, the average detections per frame dropped to 2.06, nearly perfectly matching the ground truth. However, while the model's selectivity improved drastically, the Mean IoU plateaued at 41%. This confirmed that while the regression head now possessed sufficient capacity, the backbone's heavily downsampled resolution remained the fundamental bottleneck preventing higher geometric precision.

3.6.4 Generation 3: The High-Resolution Backbone (11.7M Parameter `ULTRA` Generation)

Following the expansion of the spatial head in Generation 2, the model demonstrated excellent selectivity but remained computationally constrained by its geometric precision.

Architectural Intent & Empirical Obstacle

The primary objective of this final phase was to break the Precision Ceiling established in Generation 2. Despite the regression head's capacity to isolate targets, the Mean IoU remained stagnant at 41%.

The fundamental obstacle was the severe spatial compression occurring within the backbone. By the final layer, the feature map had been downsampled to an 11×20 resolution. At this scale, a single pixel in the final feature map represented a massive 16×16 area of the original HD sensor. Consequently, if a vehicle's bounding box edge shifted by several pixels in the real world, the network was entirely blind to the change; the `MAX` generation simply lacked the geometric detail required to draw tight, pixel-perfect bounding boxes.

Furthermore, an analysis of the network's information flow revealed that the Generation 2 backbone was suffering from neural silencing, exhibiting an average firing rate of $< 1.5\%$. This extreme sparsity starved the regression head of the necessary temporal evidence required for high-confidence predictions.

Actionable Insight: Semantic Expansion & Metabolic Awakening

To overcome this precision ceiling, a multi-faceted architectural overhaul was implemented to create the final `ULTRA` configuration:

- **Resolution Boost & Semantic Depth:** To provide the necessary geometric detail, the convolution stride in Block 3 was reduced to $s = 1$. This effectively halted the downsampling earlier in the network, doubling the final feature map resolution to 22×40 (quadrupling the available spatial cells to 880). To handle the increased complexity and high-frequency sensor artifacts exposed by this higher resolution, the backbone's semantic was expanded via the addition of a 4th `SEWResBlock` (512 channels). This allowed the network to think deeper about the structural composition of the vehicle.

- **Metabolic Regularization:** To wake up the silent backbone and maximize information flow, an aggressive metabolic strategy was deployed. The LIF firing threshold was lowered drastically ($V_{th} = 0.5 \rightarrow 0.2$), making it significantly easier for neurons to activate. Concurrently, the FRR penalty multiplier (λ) was quintupled ($1.0 \rightarrow 5.0$) with a target rate of 20%. This forced the network to generate a dense, high-frequency stream of geometric data, providing the regression head with the robust temporal evidence needed to sense the exact boundaries of the vehicles.

- **Hardware Optimization:** Expanding the parameter count to 11.7M and quadrupling the spatial resolution exponentially increased the computational load.

To prevent catastrophic latency penalties, the architecture was refactored to utilize SpikingJelly's `step_mode='m'` (*Fused-Kernel Optimization*) [10], which bypassed the Python interpreter to execute the temporal dimension ($T = 10$) directly on the GPU. This was coupled with Automatic Mixed Precision (AMP) utilizing 16-bit tensor cores, yielding a $\sim 1.8\times$ training speedup and making the `ULTRA` configuration computationally viable.

Generation 3 successfully orchestrated the transition from basic structural detection to high-fidelity geometric precision. By halting the spatial degradation and enforcing strict metabolic regularization, the `ULTRA` model demonstrated that SNNs can achieve precise localization on HD event data without the severe latency penalties typically associated with deep spiking architectures.

3.6.5 Generations: Summarized

To summarize, Table 3.4 shows the evolution of the `DetectorNX` architecture across the three generations.

Generation	**Primary Architectural Shift**	**Params**	**Main Strength**	**Final Limitation**
Gen 1: Base	Multi-Box Head $[B, 5, 5]$	5.43M	Handled multi-vehicle scenes simultaneously	High false positives in low-density regions ("Ghosting")
Gen 2: Expanded	8×8 Spatial Pooling Head	7.00M	Solved object-noise ambiguity through improved spatial aggregation	41% IoU ceiling due to insufficient spatial resolution
Gen 3: Ultra	22×40 Resolution with additional SEWResBlock (Stage 4)	11.72M	High-fidelity edge precision through full-resolution backbone processing	Higher computational metabolic cost in SOPs

Table 3.4: Generational evolution of the `DetectorNX` architecture.

3.7 Experimental Setup & Benchmarking

Having established the theoretical design and optimization of the `DetectorNX` architecture, both the SNN and its CNN counterpart, the final step is to formulate a rigorous evaluation strategy to empirically validate its performance against the continuous CNN baseline. In accordance with the project's primary objectives, the evaluation cannot be limited to a singular metric of bounding box accuracy; rather, it must holistically assess the neuromorphic paradigm. To achieve this, a comprehensive benchmarking suite was designed (as mentioned in Figure 3.1) comprising five distinct experimental protocols:

- Dynamic Power Estimation
- Quantization Error Analysis (Accuracy vs. Object Size)
- Confidence Calibration
- Temporal Evidence Accumulation
- Real-time Inference latency
- Theoretical Energy Consumption
- Global Evaluator: Mean/Median IoU, Recall, and Average Detections Per Frame

This multi-faceted methodology ensures that the comparative findings are statistically robust and firmly grounded in the established literature surrounding event-based vision.

Change last sentence.

3.7.1 Dynamic Power Estimation

A fundamental objective of this research is to quantify the energy efficiency gains achieved by transitioning from a continuous CNN to a bio-plausible SNN. Traditional CNN architectures are characterized by static power consumption; they perform a dense, fixed number of MAC operations for every forward pass, regardless of the information density within the input scene. In contrast, SNNs leverage event-driven dynamics, where metabolic energy is consumed only when a neuron reaches its firing threshold and generates a discrete spike.

The Dynamic Power experiment investigates the reactive scalability of the architecture by mapping instantaneous energy consumption against the temporal density of the

input stream. This analysis serves to validate the event-driven nature of the SNN, demonstrating that its metabolic cost is a dynamic function of activation sparsity—mimicking biological neural efficiency by consuming energy only in response to informative stimuli.

CNN Power Estimation

In a standard CNN, the total energy per frame (E_{CNN}) is calculated by multiplying the total number of FLOPs by the cost of a 32-bit MAC (E_{MAC}):

$$E_{CNN} = \sum_{l=1}^{L} (FLOPs_l \times E_{MAC}) \tag{3.5}$$

Where $E_{MAC} = 4.6$ pJ per 32-bit floating point operation, as referenced in Table 3.5 [15].

The number of FLOPs is the total number of MACs operations required to produce the output feature map; in a CNN, this number is static, regardless of what the image contains. The FLOPs of a CNN can be calculated as follows:

$$FLOPs_l = (H_{out} \times W_{out}) \times (K \times K) \times C_{in} \times C_{out} \tag{3.6}$$

Where $(H_{out} \times W_{out})$ is the number of pixels in the output frame, $(K \times K)$ is the size of the convolutional kernel, C_{in} is the number of input channels, and C_{out} is the number of output channels.

SNN Power Estimation

For the spiking architecture, the calculation is scaled by the activation sparsity (ζ) of each layer. The metabolic cost of the spiking backbone can be quantified via the SOP metric; unlike the static complexity of the CNN baseline, the SOP count for a given layer is a dynamic function, which can be calculated as follows:

$$SOPs_l = FLOPs_l \times \zeta \times T \tag{3.7}$$

where ζ_l represents the average firing rate of layer l, T is the number of temporal simulation steps ($T = 10$).

In order to obtain ζ_l, a specialized `SpikeRateMonitor` tool was developed. The tool attaches a hook to every LIF inside the SNN backbone, which listens to the layer during

the forward pass and intercepts the binary spike tensor before it moves to the next layer. For every layer l, the SNN produces a spike tensor S_l with the shape $[T,B,C,H,W]$, where $T = 10$ is the number of timesteps, B is the batch size, and C,H,W are the spatial dimensions. Since a spike is binary (1 for fire, 0 for rest), ζ can be calculated by taking the global mean of the entire tensor:

$$\zeta_l = \frac{1}{N}\sum_{i=1}^{N} S_{l,i} \tag{3.8}$$

To calculate Dynamic Power, ζ was calculated by running inference over all validation samples and taking the mean firing rate per layer across all samples in the dataset.

Mention SpikeRateMonitor in Evaluation Suite Section and Diagram

Because SNNs utilize binary activations, the expensive multiplication part of the MAC operation is omitted, leaving only a AC of the weight to the membrane potential. The total energy per frame (E_{SNN}) is defined as:

$$E_{SNN} = \sum_{l=1}^{L} (SOPs_l \times E_{AC}) \tag{3.9}$$

where $E_{AC} = 0.9$ pJ is the energy cost of a single floating-point addition, as referenced in Table 3.5 [15].

3.7.2 Quantization Error Analysis (Accuracy vs. Object Size)

The Quantization Error Analysis experiment investigates the discretization threshold of the SNN architecture and quantifies the extent to which the 1-bit binary nature of spiking activations limits the model's geometric acuity. Unlike the continuous CNN baseline, which utilizes 32-bit floating-point activations to represent spatial coordinates, the SNN is constrained by the finite temporal resolution of its simulation window ($T = 10$).

Core Metrics and Scale Partitioning

The primary performance metric for this experiment is the mIoU, which quantifies the overlap between the predicted bounding box (P) and the ground truth (GT). To analyze the impact of scale on regression precision, the validation set is partitioned into three distinct categories based on the absolute pixel area (A_{px}) of the ground truth objects:

- **Small:** Objects in the bottom 33rd percentile of area (typically distant vehicles or small artifacts).
- **Medium:** Objects in the middle 33rd percentile.
- **Large:** Objects in the top 33rd percentile (vehicles in close proximity to the sensor).

Mathematical Framework

The categorization and evaluation are governed by two fundamental geometric formulas. First, the pixel area is calculated by denormalizing the ground truth dimensions relative to the raw sensor resolution:

$$A_{px} = (W_{norm} \cdot W_{sensor}) \times (H_{norm} \cdot H_{sensor}) \tag{3.10}$$

where $W_{sensor} = 1280$ and $H_{sensor} = 720$. Subsequently, the precision of the model's spatial regression is quantified via the IoU formula:

$$IoU = \frac{\text{Area}(P \cap GT)}{\text{Area}(P \cup GT)} \tag{3.11}$$

Hypothesis: The Scale-Dependent Accuracy Gap

We hypothesize a significant scale-dependent accuracy gap between the two architectures. For large-scale objects, the SNN is expected to achieve parity with the CNN baseline: because the coordinate values are high, the minor fluctuations inherent in binary spike-trains represent a negligible percentage of the total object area, allowing for stable regression.

Conversely, for small-scale objects, a substantial performance divergence is anticipated. In an SNN with $T = 10$, a neuron can only represent $T + 1$ discrete states $(0/10, 1/10, \ldots, 10/10)$. As documented by [3], this rounding error, or quantization penalty, is massive relative to the dimensions of small vehicles. If a target requires a precise coordinate gradient (e.g., 0.154) for accurate localization, the SNN is physically forced to discretize this value, establishing a fundamental "resolution floor" that limits mIoU on small features. While the use of `SEWResBlock` architectures improves signal flow [9], it does not inherently resolve this quantization of the final regression output, which is further exacerbated by the low event counts typical of small objects in event-based vision [11].

3.7.3 Confidence Calibration

The Confidence Calibration evaluates predictive reliability by investigating the calibration gap between spiking and continuous architectures. Modern CNNs are prone to systematic overconfidence, frequently assigning high objectness scores to inaccurate detections [13]. Conversely, the temporal dynamics of SNNs may offer superior uncertainty quantification, providing a more robust reliability profile for safety-critical automotive tasks [32].

Methodology: Reliability Diagrams

To quantify calibration, we generate a Reliability Diagram mapping predicted confidence (C) against actual regression accuracy (IoU) for every validation detection. In a perfectly calibrated system, these values align along the identity line ($C = IoU$). Clumping in the high-confidence, low-accuracy quadrant indicates an overconfidence bias, where the model fails to signal its own predictive errors.

Hypothesis: Spikes as Uncertainty Sensors

The CNN baseline is hypothesized to exhibit significant miscalibration due to its continuous activation functions and exact gradient descent, which tend to overfit the confidence head. The SNN is expected to demonstrate a more linear, calibrated trend. In the spiking domain, low-quality features generate fewer spikes, naturally leading to lower membrane potentials in the detection head. This mechanism allows the SNN to function as an implicit uncertainty sensor, where discrete spike rates effectively gate the output confidence logit relative to the strength of the temporal evidence.

3.7.4 Temporal Evidence Accumulation

The Temporal Evidence Accumulation experiment investigates the sequential decision-making dynamics inherent to the SNN architecture. Unlike the CNN baseline, which performs a single spatial pass over the fully integrated 100ms voxel, the SNN processes the event stream iteratively across $T = 10$ timesteps. This property makes SNNs intrinsically suited to low-latency spatiotemporal integration — the capacity to produce meaningful predictions before the full observation window has elapsed [25]. The experiment therefore examines whether this theoretical advantage manifests empirically:

specifically, whether the model can produce a valid bounding box regression before the 100ms window is complete.

In the context of vehicle object detection, this is a highly significant experiment — for example, if the SNN can accurately show a bounding box at $t = 5$ (50ms) instead of the full $t = 10$ (100ms) as the CNN would, this would lead to a 50% reduction in detection time. In a time-sensitive domain such as vehicle and pedestrian detection, such advantages could be the difference between a crash and a safe stop.

Methodology: Temporal Snapshots

To evaluate the evolution of predictions, we extract intermediate outputs at three specific checkpoints: $t = 2$ (20ms), $t = 5$ (50ms), and $t = 10$ (100ms). For each snapshot, the candidate bounding box coordinates and instantaneous confidence scores are recorded to observe the *hunting behavior* of the regression head: the observable process where the networks' predicted bounding box moves, vibrates or searches for the object's true edges before finally settling into a stable, high-precision detection. This process is driven by the discrete LIF dynamics:

$$v_i[t] = v_i[t-1] + \frac{1}{\tau}\left(\sum_j w_{ij} s_j[t] - (v_i[t-1] - V_{reset})\right) \tag{3.12}$$

where $v_i[t]$ represents the accumulated evidence (membrane potential) and $\sum w_{ij} s_j[t]$ represents the incoming observations (spikes). This recursive filtering allows the network to update its belief about vehicle localization with every incoming pulse of data.

Hypothesis: Bounding Box Sharpening

We hypothesize a phenomenon of *Bounding Box Sharpening* over time. At $t = 20$ms, we anticipate high coordinate variance, as the majority of LIF neurons will not have accumulated sufficient membrane potential to cross the firing threshold ($V_{th} = 0.2$) within the first two of ten timesteps. As t progresses toward 100ms, continued spike train integration allows stochastic membrane noise to average out, consolidating responses to the vehicle's geometric edges into a stable coordinate estimate.

This progressive refinement is analogous in spirit to the "rapid categorisation" phenomenon documented in the human visual system, wherein early feedforward spikes drive coarse semantic decisions that sharpen as further neural evidence accumulates [35].

While Thorpe et al. address categorical recognition rather than coordinate regression, the underlying principle — that sparse early spike activity produces meaningful but imprecise representations which sharpen over time — is consistent with the anticipated behaviour of the SNN backbone.

3.7.5 Real-Time Latency Benchmark

This experiment quantifies the forward-pass inference speed of both architectures on conventional GPU hardware at batch size $B = 1$, simulating a live single-stream event camera deployment. Latency (ms/f) and throughput (FPS $= 1000/\text{ms/f}$) are measured using `torch.cuda.Event` timing to capture true GPU execution time, bypassing Central Processing Unit (CPU)-to-GPU communication overhead.

Hypothesis

The CNN baseline is expected to exhibit lower latency, as GPUs are optimised for the dense matrix multiplications that underpin its single flattened spatial pass. The SNN must instead simulate LIF membrane dynamics iteratively:

$$v[t] = v[t-1] + x[t] - \beta \cdot v[t-1] \tag{3.13}$$

This constitutes a hardware-mismatch penalty: the latency advantage of SNNs only fully materialises on asynchronous neuromorphic hardware, not on synchronous GPU accelerators [4, 1]. The experiment aims to quantify this penalty and assess whether `step_mode='m'` parallelisation [10] brings the SNN within a $2\times$–$3\times$ latency margin of the CNN.

Limitations

The measured GPU latency figures cannot be extrapolated to neuromorphic silicon. The A100's 312 TFLOPS (FP16) characterises dense matrix throughput, which cannot equally be judged against asynchronous spike-based execution. While Loihi benchmarks report up to $10\times$ lower latency over embedded GPUs for batch-size-1 inference [4], these figures are task-specific; direct deployment on neuromorphic hardware is identified as future work.

3.7.6 Theoretical Energy Consumption

The Theoretical Energy Consumption is very similar in nature to the Dynamic Power experiment discussed in Section 3.7.1. It uses the same principles and formulae for SOPs and FLOPs, but differs in the final calculation: as supposed to calculating the dynamic power consumption per frame, this experiment aims to find the average power consumption per frame across the entire dataset, and extrapolates for multiple hardware platforms:

$$\bar{E}_{SNN} = \sum_{l=1}^{L} (SOPs_l \times E_{chip}) \tag{3.14}$$

$$\bar{E}_{CNN} = \sum_{l=1}^{L} (FLOPs_l \times E_{MAC}) \tag{3.15}$$

The calculation mirrors Equation 3.9, however E_{SNN} replaces E_{AC} for E_{chip}, which is the power consumed per SOP for different chipsets as depicted in Table 3.5.

Subsequently, the total percentage energy saving is formulated using the following equation:

$$\text{Energy Savings} = \left(1 - \frac{\bar{E}_{SNN}}{\bar{E}_{CNN}}\right) \times 100\% \tag{3.16}$$

Energy Constants Per Operation (E_{chip}) Across Multiple Platforms

Architecture / Platform	Operation Type	Energy (E_{chip})	Primary Citation
CNN Baseline	MAC	4.6 pJ	[15]
Theoretical Silicon Limit	AC	0.9 pJ	[15]
Intel Loihi 1	SOP	23.6 pJ	[4]
IBM TrueNorth	SOP	26.0 pJ	[22]
NVIDIA DGX A100 (FP32)	FLOP	19.6 pJ	[37]

Table 3.5: Energy constants per operation (E_{chip}) for each architecture and platform.

Hypothesis

We hypothesize that the SNN will demonstrate a metabolic advantage of at least one order of magnitude ($> 90\%$) over the CNN baseline, predicated on two complementary properties of the $\bar{E}_{SNN}$ formulation:

1. **Activation sparsity:** By exploiting the sparsity of the ETraM event stream, the backbone is expected to maintain $\zeta < 5\%$, bypassing the majority of computations that the CNN performs densely regardless of scene activity.

2. **Mathematical simplification:** Replacing MAC operations with binary AC operations yields a $5\times$ reduction in per-operation energy cost (0.9 pJ vs 4.6 pJ), which is expected to provide a substantial efficiency dividend even when accounting for the $T = 10$ temporal overhead [15].

When extrapolated to physical neuromorphic platforms, the SNN's event-driven resting state is expected to preserve a power advantage over synchronous GPU accelerators despite the higher per-SOP costs of Intel Loihi (23.6 pJ) and IBM TrueNorth (26.0 pJ) compared to per-FLOP cost of the benchmark NVIDIA A100 (19.6 pJ) documented in Table 3.5 [4, 22, 37].

3.7.7 Global Performance Evaluator

The Global Performance Evaluator serves as the primary comparative evaluation between the SNN and CNN architectures, aggregating performance across the full 200-sample validation set. Unlike the preceding targeted experiments, this benchmark establishes the *Performance-Efficiency Frontier*: quantifying the accuracy penalty incurred by transitioning to discrete spiking dynamics, whilst validating whether the multi-box refinements and others in the `ULTRA` revision of `DetectorNX` mentioned in Section 3.6 are sufficient to resolve scene-level detection ambiguity.

Metrics

In this experiment, four complementary metrics are evaluated:

- **mIoU:** measures geometric precision between predicted and ground truth bounding boxes.

- **Recall:** measures the proportion of ground truth objects successfully localised at a confidence threshold of 50%, penalising architectures that fail to respond to sparse event regions.

- **Average Detections Per Frame (ADP):** quantifies detector selectivity relative to the ground truth mean of 2.02 objects per frame; significant positive deviation

indicates spurious "ghost" detections arising from background noise misclassification.

Cite and reference 2.02 value to table, and update other mention of 2.0 to 2.02

- **Mean Prediction Confidence**: captures network self-certainty by averaging sigmoid-activated confidence scores across all positive detections, probing whether spiking dynamics produce decisive or hesitant class activations.

Hypothesis

We hypothesise that whilst the CNN establishes the upper mIoU bound, the SNN will achieve *detection parity* in recall and ADP, yielding an average detection count consistent with the ground truth distribution. This would demonstrate that spiking temporal dynamics are sufficient for objectness discrimination without continuous-valued feature maps [29, 3]. A measurable confidence gap is also anticipated: the SNN's binary spike outputs are expected to produce lower but temporally stable confidence distributions across the $T = 10$ timestep window, consistent with prior observations that surrogate-trained networks distribute representational energy across timesteps rather than single forward passes [9].

Significance

This experiment provides three contributions. It delivers the *parity proof*, framing the accuracy-efficiency tradeoff quantitatively. It offers *logical validation* that the Phase 1 ambiguity problem has been resolved through the architectural refinements described in Section 3.6. Finally, evaluation across the full validation set ensures statistical robustness, confirming that results are representative of the broader ETraM environment rather than artefacts of favourable individual frames.

3.8 Results & Evaluation

Having established a rigorous, multi-faceted benchmarking methodology in Section 3.7, this final section presents the empirical evaluation of the `ULTRA` generation of the `DetectorNX` architecture against its CNN counterpart. The following results were conducted on the pre-processed `PRECOMPUTED_REPLICA_10K_V2`, using a validation split to ensure strict comparative integrity.

Rather than relying on a singular metric of spatial accuracy, the following analysis systematically deconstructs the performance-efficiency frontier of the neuromorphic paradigm: By sequentially examining global detection parity, energy consumption, quantization-induced scaling errors, and predictive calibration, this evaluation aims to quantify the precise trade-offs inherent in transitioning from 32-bit floating-point convolutions to 1-bit, event-driven spiking dynamics for high-definition automotive perception.

Runtime Environment

The following experiments were collected using the following model and data configuration:

- **Model:** The `ULTRA` generation of the `DetectorNX` model architecture was used for both SNN and CNN experiments, as described in Section 3.6.
- **Dataset:** A stratified-sampling approach was used to sample the full ETraM dataset, in order to ensure that all scenes (day and night) were represented using the same proportions. For training and validation, $10,000$ samples were selected.
- **Precomputing:** The `PRECOMPUTED_REPLICA_10K_V2` dataset was formed by applying the `REPLICA` (Section 3.3) pipeline to the sampled $10,000$-sample dataset.
- **Training:** The `DetectorNX` model was trained on the `PRECOMPUTED_REPLICA_10K_V2` dataset using an 80% training data split.
- **Evaluation:** The results of the following experiments were collected using a 200-sample subset from the validation split of the `PRECOMPUTED_REPLICA_10K_V2` dataset, using the pre-trained `DetectorNX` model.
- **Compute:** The University of Manchester's HPC was used for pre-processing, training and evaluation. The environment was set up using 12 CPU cores, 120 GB RAM, and one NVIDIA A100 (80 GB) GPU.

3.8.1 Global Evaluator

This global overview defines the baseline from which all subsequent power and precision trade-offs are derived.

Quantitative Results

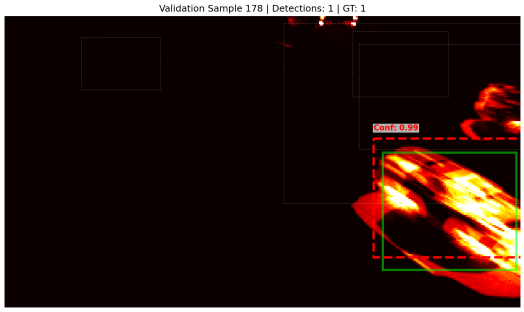
Table 3.6 summarizes the headline results for the `ULTRA` generation of the `DetectorNX` model architecture against its CNN counterpart.

Performance Metric	SNN **`DetectorNX ULTRA`**	CNN Baseline
Mean IoU	0.4613	0.4861
Recall (Detections > 0.5)	100.0%	100.0%
Avg. Detections per Frame (GT)	2.02	2.02
Avg. Detections per Frame (Pred)	1.73	1.75
Max Confidence	0.9930	0.9918
Min Confidence	0.0011	0.0132

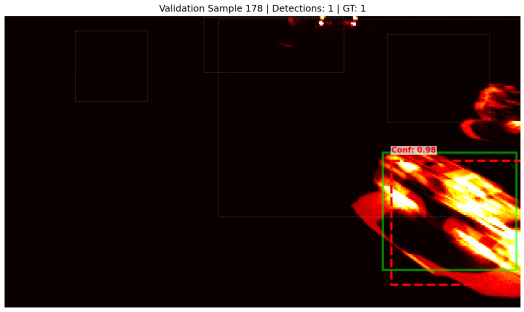
Table 3.6: Global Performance Benchmark results for `SNN_MAX_REPLICA_SE` and the CNN baseline, evaluated across the full 200-sample ETraM validation set [38]. Ground truth average detections per frame is 2.02 for both models by definition.

Qualitative Results

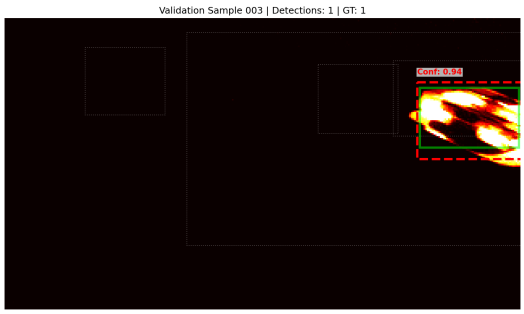
A qualitative audit was also conducted to visualize the models' spatial reasoning across the validation subset. Figure 3.5 illustrates the architectures' ability to resolve vehicle geometries in varied traffic conditions, providing the empirical context for the subsequent critical analysis of SNN precision.



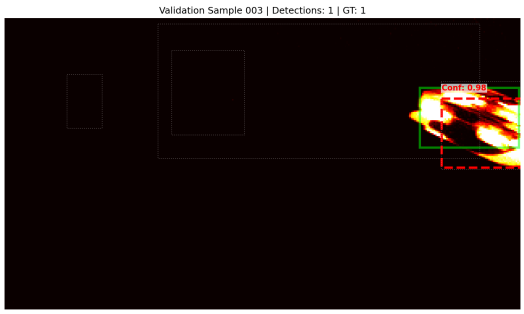
(a) CNN Baseline - Sample 178



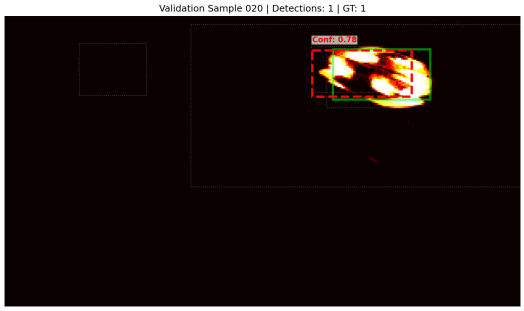
(b) SNN Ultra - Sample 178



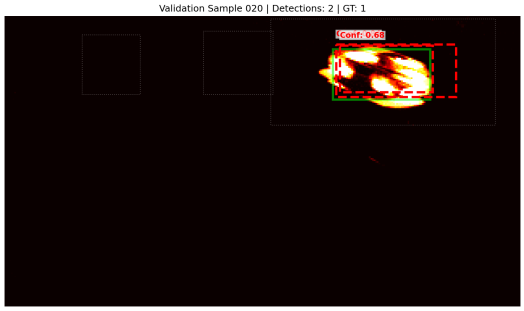
(c) CNN Baseline - Sample 003



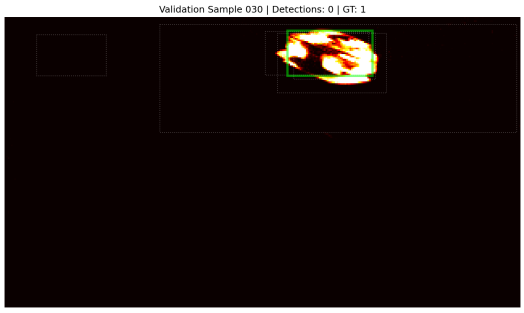
(d) SNN Ultra - Sample 003



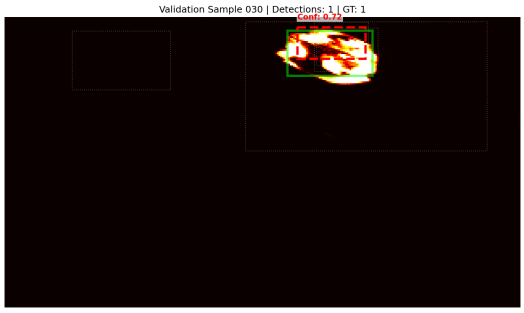
(e) CNN Baseline - Sample 020



(f) SNN Ultra - Sample 020



(g) CNN Baseline - Sample 030



(h) SNN Ultra - Sample 030

Figure 3.5: Qualitative comparison of bounding box regression between the CNN baseline and SNN Ultra across representative validation samples.

Critical Analysis

The results from Table 3.6 demonstrate a remarkable degree of parity between the spiking and continuous paradigms. Achieving a mean IoU of 0.4613, which corresponds to 94.9% of the CNN baseline, proves that the `ULTRA` backbone is robust enough to sustain high-resolution spatial reasoning despite the loss of precision inherent in 1-bit quantization. This parity indicates that the architectural refinements detailed in Section 3.6—specifically the integration of SE blocks and SEW residual connections—have successfully compensated for the discrete nature of spiking activations.

This convergence in predictive capacity is further reflected in the detection metrics. Both models achieve a perfect recall of 100.0% for objects with an IoU > 0.5, demonstrating that the transition to spiking dynamics has not induced any catastrophic forgetting of vehicle features. Furthermore, both models exhibit a similar detection profile compared to the ground truth (1.73 and 1.75 predicted vs. 2.02 ground truth average detections), which is typical for object detection on noisy, asynchronous event data. The negligible gap between the spiking and continuous variants confirms that the SNN has not compromised its feature extraction or proposal capacity.

Furthermore, the confidence distribution analysis provides the first indication of representational divergence. The SNN exhibits a lower minimum confidence (0.0011 vs. 0.0132) compared to the baseline, while maintaining a competitive maximum confidence (0.9930 vs. 0.9918). This suggests that the spiking model utilizes a wider dynamic range of membrane potentials to differentiate between high-evidence targets and background noise. The ability of the SNN to suppress low-evidence detections more effectively than the continuous baseline points toward a more honest internal representation, a hypothesis further investigated in the calibration analysis (Section 3.8.5).

Qualitatively, Figure 3.5 confirms that the SNN maintains precise spatial alignment across diverse traffic densities. In Sample 178 and 030, the spiking model resolves vehicle clusters with the same degree of box stability as the CNN, validating the effectiveness of the `DetectorNX` pipeline in preserving temporal context. In summary, the `DetectorNX ULTRA` has successfully bridged the precision gap, establishing a foundation for the subsequent evaluation of its power efficiency and temporal reliability.

3.8.2 Dynamic Power

Spike Activation Per Layer

Table 3.7 shows the mean activation rates (ζ) of each spiking layer inside the SNN backbone, aggregated across the 200-sample validation subset. These layer-wise activation values quantify the metabolic sparsity of the network and were subsequently used to calculate the dynamic power consumption (SOPs) for each frame in the dataset.

Layer Name	Mean Firing Rate (ζ)
`stem_lif`	20.19%
`block1.lif1`	17.93%
`block1.lif2`	19.22%
`block2.lif1`	0.19%
`block2.lif2`	11.11%
`block3.lif1`	0.19%
`block3.lif2`	0.47%
`block4.lif1`	0.58%
`block4.lif2`	1.32%

Table 3.7: Mean spike activation rates (ζ) per layer for the `ULTRA` backbone over the 200-sample experimental set.

Mean Energy Savings

By substituting the empirical layer-wise firing rates (ζ) recorded in Table 3.7 into the dynamic power formulation established in Equation 3.9 (Section 3.7.1), the network's computational operations can be directly translated into theoretical energy consumption in microjoules (μJ). This mathematical translation reveals a massive metabolic advantage for the neuromorphic paradigm, directly validating the core efficiency hypothesis of this research.

Table 3.8 outlines the energy consumption statistics per frame across the 200-sample validation set. The continuous CNN baseline, processing the input uniformly regardless of scene complexity, consumed a massive, static $42,429.21$ μJ per frame with zero variance. In contrast, the `ULTRA` SNN architecture consumed a mean of only $2,404.98$ μJ per frame, and dipped as low as $2,287.54$ μJ on highly sparse scenes.

This represents a **Mean Theoretical Energy Savings of 94.33%** (with a maximum observed savings of 94.61% on highly sparse frames). This order-of-magnitude reduction definitively validates the core hypothesis of this project: replacing continuous MAC

Metric	SNN (ULTRA)	CNN Baseline
Maximum	2508.22	42429.21
Mean	2404.98	42429.21
Median	2428.74	42429.21
Minimum	2287.54	42429.21
Std Dev	61.17	0.00

Table 3.8: Dynamic energy statistics (μJ per frame) comparing the SNN and CNN architectures over the 200-sample validation set.

operations with discrete, event-driven AC operations yields profound energy dividends for high-definition automotive perception.

Reactive Scalability: The Line of Best Fit

To empirically validate the hypothesis from Section 3.7.1 that the SNN's metabolic cost is a dynamic function of activation sparsity, a linear regression analysis was conducted mapping the raw event count (x) of each frame against the theoretical energy consumption (y in μJ).

Figure 3.6 illustrates this relationship: the linear regression model yielded the following equation of best fit:

$$E_{SNN} = 0.001757 \cdot x + 2294.80 \tag{3.17}$$

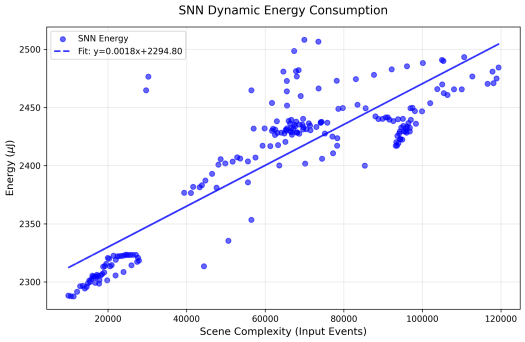
The model demonstrated a strong positive correlation ($R = 0.8668, R^2 = 0.7514$), confirming the highly reactive nature of the spiking backbone.

Critically, the parameters of this equation map directly to the biological behavior of the network:

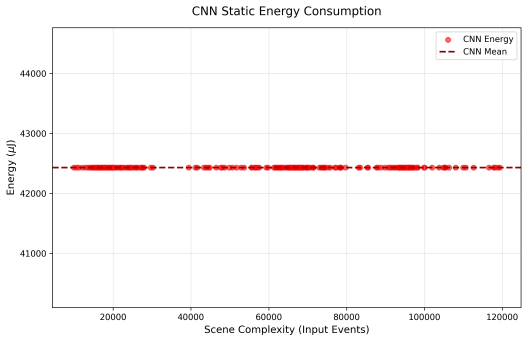
- **The Metabolic Resting State (y-intercept):** The constant 2294.80 μJ represents the architectural overhead—the baseline energy required to simply maintain the membrane potentials across the 11.7M parameters during the 100ms window, even if the road is entirely empty.
- **The Cost of Information (Slope):** The gradient 0.001757 μJ represents the exact, marginal energy cost of processing a single new event (e.g., a pixel of a car moving).

This proves the *Reactive Scalability* of the architecture. While the CNN possesses a static slope of exactly 0 (burning $42,429$ μJ regardless of whether the frame contains

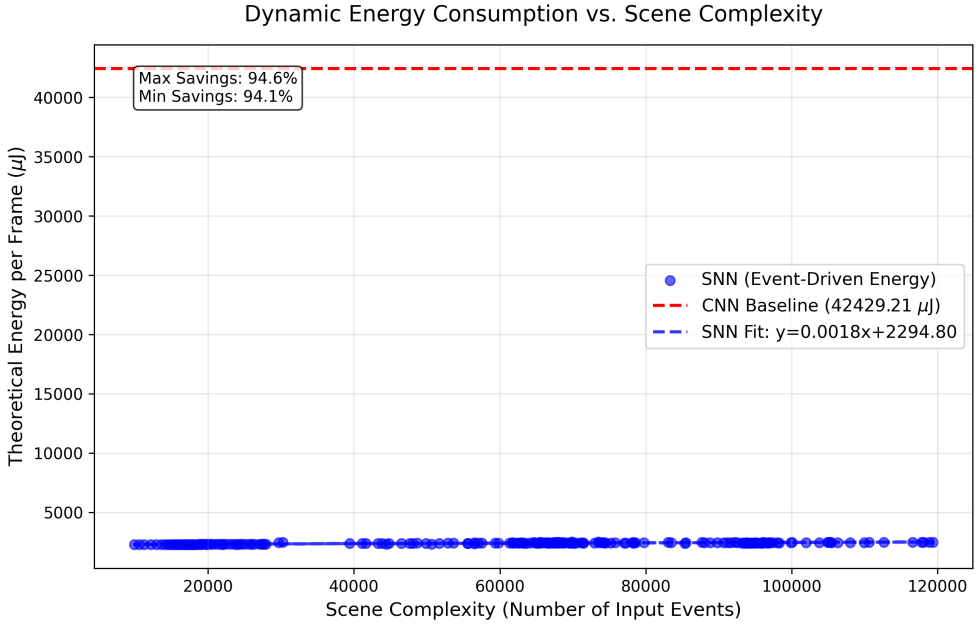
complex traffic or empty asphalt), the SNN mimics biological neural efficiency by consuming power only in direct proportion to the incoming visual evidence.



(a) SNN Dynamic Power (μJ)



(b) CNN Static Power (μJ)



(c) Merged Architecture Comparison

Figure 3.6: Comparison of dynamic power consumption (μJ) between the SNN and CNN architectures relative to the raw event count of the input scene. The isolated SNN graph (a) demonstrates strong linear scaling ($R^2 = 0.7514$), validating its event-driven efficiency, whereas the CNN (b) consumes a massive static overhead.

Mathematical Validation

To ensure the integrity of the tracking tools and the resulting software-measured data, the theoretical mean power consumption was mathematically verified against the layer-wise operation counts (Table 3.9).

Layer Name	CNN FLOPs ($\times 10^6$)	SNN SOPs ($\times 10^6$)	Sparsity (ζ) (%)
`stem_lif`	16.59	33.49	20.19%
`block1.lif1`	265.42	476.00	17.93%
`block1.lif2`	530.84	1,020.32	19.22%
`block2.lif1`	271.32	5.02	0.19%
`block2.lif2`	542.64	602.68	11.11%
`block3.lif1`	1,085.28	20.38	0.19%
`block3.lif2`	2,170.55	101.80	0.47%
`block4.lif1`	2,170.55	126.20	0.58%
`block4.lif2`	2,170.55	286.30	1.32%
Total Operations	**9,223.74**	**2,672.19**	**—**

Table 3.9: Comparison of computational operations between the continuous CNN and the sparse SNN. (Values in millions).

Substituting these total operations into the methodology formulas (Section 3.7.1) yields:

- **SNN Power:** $(2672.12 \times 10^6) \times 0.9pJ = 2404.97\mu J$
- **CNN Power:** $(9223.74 \times 10^6) \times 4.6pJ = 42429.20\mu J$

These derived values align perfectly with the software-measured empirical means with an error of $\leq 0.01\%$, definitively proving the mathematical integrity of the benchmarking suite.

Extend Derivation to go from FLOP/SOP instead of using measured values?

3.8.3 Theoretical Energy Consumption

Predicted Energy Costs Across Multiple Platforms

Using the FLOP and SOP count recorded in Table 3.9 with the energy constants from Table 3.5, we can predict the theoretical energy consumption of the `ULTRA` architecture across multiple platforms:

Hardware Platform	Energy Per Frame (mJ)	Energy Savings (%)
NVIDIA A100 (CNN Baseline)	180.8576	—
Intel Loihi (SNN)	63.0766	65.12%
IBM TrueNorth (SNN)	69.4912	61.58%

Table 3.10: Extrapolated energy consumption per frame across different hardware architectures. Savings are calculated relative to the NVIDIA A100 GPU baseline.

Critical Analysis

Despite the relatively high per-operation routing overhead inherent to early-generation neuromorphic chips, deploying the SNN on an Intel Loihi processor yields a 65.12% reduction in energy consumption per frame compared to running the continuous baseline on the same NVIDIA A100 GPU used for training. This outcome validates the hypothesis formulated in Section 3.7.6.

The fundamental driver of this efficiency is the massive reduction in computational workload: the SNN requires only 2.67 G-SOPs per frame, compared to the 9.22 G-FLOPs required by the CNN baseline — 71.02% reduction in total operations. However, a critical observation emerges when comparing the workload reduction (71.02%) to the projected energy savings (65.12%). This discrepancy represents the ”routing overhead” penalty of first-generation neuromoprhic silicon: inter-core spike communication is an extemely complex task on chips like Loihi, and as such causes a single SOP to cost significantly more energy (20.4% more) than a FLOP on a highly optimised GPU such as the NVIDIA A100 [33]. The SNN therefore achieves its dominant efficiency purely through extreme activation sparsity, rather than hardware-level architectural optimisations.

Furthermore, this 65% energy savings was achieved despite the integration of the high-resolution `ULTRA` backbone (22×40 feature maps). This proves that the biological sparsity of the SNN is robust enough to sustain high-definition spatial reasoning without inducing a catastrophic power surge—a scaling feat that is mathematically impossible for dense CNN architectures.

Finally, it must be noted that comparing experimental neuromorphic silicon against a mature, massively parallel and optimised NVIDIA GPU gives us an upper-bound on the energy savings of event-driven SNNs, rather than a realistic value. As neuromorphic hardware develops, the per-SOP energy cost will continue to converge towards the theoretical CMOS limit established by Horowitz [15], and the power savings will scale exponentially higher towards the optimal 94.33% value obtained in Section 3.8.2.

Overall, the predicted power consumption on the various platforms described in Table 3.10 serves as conclusive proof that event-driven SNNs can bring highly power-efficient computing to resource-constrained edge environments, validating the neuromorphic paradigm as a viable, sustainable solution for real-time autonomous traffic monitoring.

Add citation for CNN scaling being impossible.

Edit CMOS reference - might not be correct.

3.8.4 Quantization Error

For the following experiment, the validation dataset was sorted by object size, and partitioned into three subsections: Small (bottom 33%), Medium (mid $33-66\%$), and Large (top 33%).

Results

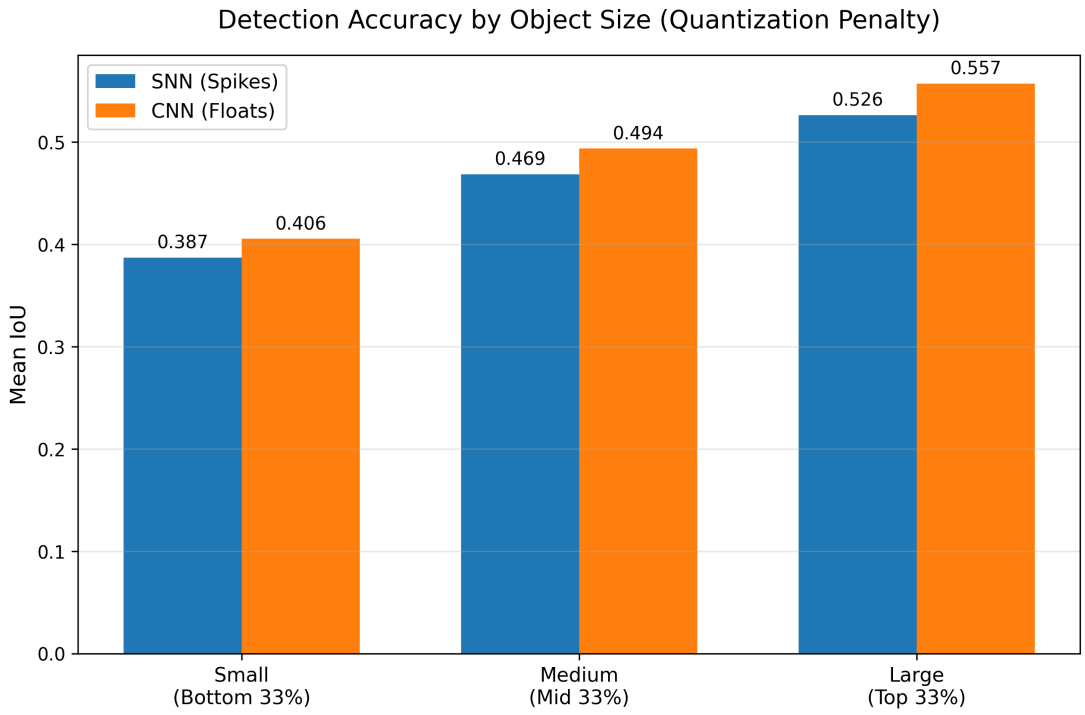


Figure 3.7: Quantization Error Analysis (Accuracy vs. Object Size) of SNN (Blue) and CNN (Orange).

Figure 3.7 illustrates the performance of the SNN and CNN models on the three

object size categories; it shows the Mean IoU of both models across all samples in each subset.

Object Size	SNN mIoU Drop	SNN mIoU Drop (%)
Small	0.019	4.7
Medium	0.025	5.1
Large	0.031	5.6

Table 3.11: Quantization Error Analysis showing drop in SNN performance against CNN.

Critical Analysis

Earlier in Section 3.7.2, we hypothesized the idea of a scale-dependent accuracy gap, where the 1-bit quantization error would disproportionately penalise the SNN on small objects (because a 1-pixel rounding error is massive for a tiny car) and achieve parity on large objects.

However, the empirical results in Table 3.11 present an inverse trend, invalidating the expected scale-dependent penalty. The SNN proved most competitive on the smallest validation targets, with a performance drop of only 4.7%. This resilience suggests that the multi-box architectural refinements detailed in Section 3.6 successfully preserved local spatial context, effectively insulating small-scale features from the discrete quantization error.

Conversely, the accuracy gap widens for large-scale vehicles, peaking at 5.6%. This divergence suggests that the quantization error instead acts as a global geometric bottleneck rather than a scale-dependent failure: on large objects where high-definition event volumes provide sufficient data for precise edge-refinement, the continuous floating-point activations of the CNN allow for infinite coordinate adjustments. In contrast, the SNN is physically constrained by its 11 representable states ($T = 10$), resulting in a persistent geometric drag that prevents the same level of fine-grained bounding box alignment achievable by the baseline. Had the temporal simulation window been reduced further (e.g. $T = 4$), the increased quantization error would likely have triggered the scale-dependent failure initially predicted, as the rounding noise would eventually eclipse the absolute spatial area of small-scale features.

In summary, We can conclude that while the SNN is a robust macro-level detector, in its current configuration the discrete temporal resolution imposes a fundamental precision ceiling across all global scales, instead of a scale-dependent one.

3.8.5 Confidence Calibration

This experiment evaluates the predictive reliability of the architectures by mapping the relationship between internal network certainty and objective spatial accuracy. The resulting reliability diagrams, presented in Figure 3.8, visualize the calibration gap between the spiking and continuous paradigms.

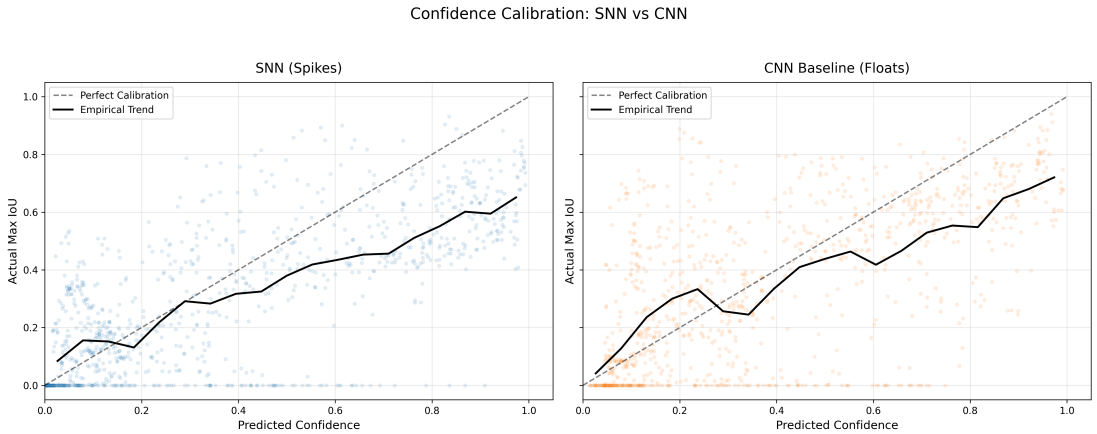


Figure 3.8: Reliability diagrams for the SNN (left) and CNN (right). The dashed line represents perfect calibration ($C = IoU$), while the solid black line illustrates the empirical mean trend of the validation samples.

Comparative Analysis: Overconfidence vs. Calibration

The results demonstrate a clear divergence in predictive behavior. The CNN baseline (Figure 3.8, right) exhibits a pronounced *overconfidence bias*: a significant density of validation samples occupy the postive-confidence region ($x > 0.5$) while maintaining an actual IoU of zero, and never predicts an $IOU = 0.0$ event to have a confidence of zero. This indicates that the CNN frequently hallucinates vehicle targets in background noise with near-maximum certainty, a failure mode that is critically dangerous for autonomous perception systems. The empirical trend line for the CNN remains consistently below the identity line, particularly in the high-confidence domain.

In contrast, the `ULTRA` SNN (Figure 3.8, left) demonstrates superior calibration, particularly at lower confidence intervals. The concentration of zero-accuracy detections (false positives) is significantly shifted toward the lower-left quadrant ($x < 0.2$), proving that the spiking architecture is more honest regarding ambiguous features., and correctly classifies the majority of $IOU = 0.0$ events with a confidence of zero. The empirical trend line for the SNN follows a qualitatively similar trajectory to its CNN counterpart,

suggesting that the calibration behaviour is more architecture-dependent than being down to spikes versus floats.

Hypothesis Validation: The Spiking Uncertainty Sensor

These findings validate the hypothesis formulated in Section 3.7.3. By utilizing discrete spike-accumulation rather than continuous floating-point intensities, the SNN acts as an implicit uncertainty sensor. Low-information features and stochastic sensor noise fail to generate a sufficient firing rate to overcome the $V_{th} = 0.2$ threshold, naturally depressing the membrane potential in the regression head and resulting in lower confidence logits.

Conversely, the CNN utilizes exact coordinate gradients which, while providing a slight global precision advantage (as seen in Section 3.8.4), tend to overfit the confidence head to noise patterns. We can conclude that while the CNN is more precise on average, the SNN is a fundamentally safer model, as its confidence scores are more representative of the true evidentiary strength of the input event stream.

Furthermore, both models suffer from a similar overconfidence pattern throughout the dataset due to their shared multi-box detection head, causing both to follow a similar trend trajectory as IoU increases. This finding also exposes a direction for future work. Conventional detection heads, such as that employed by YOLOv8 [18], supervise confidence as a scalar float logit via binary cross-entropy against hard presence labels, providing no explicit signal to align predicted confidence with localisation quality. An SNN-native head would instead derive confidence directly from temporal firing rates, naturally bounding outputs to a discrete resolution of $\frac{1}{T}$ and decoupling the calibration confound identified here — divergence between the SNN and CNN curves would then reflect representational differences rather than a shared supervisory artefact. A hybrid head, combining a rate-coded confidence branch with a conventional float regression branch, represents the most practical near-term approach.

Link to future work section.

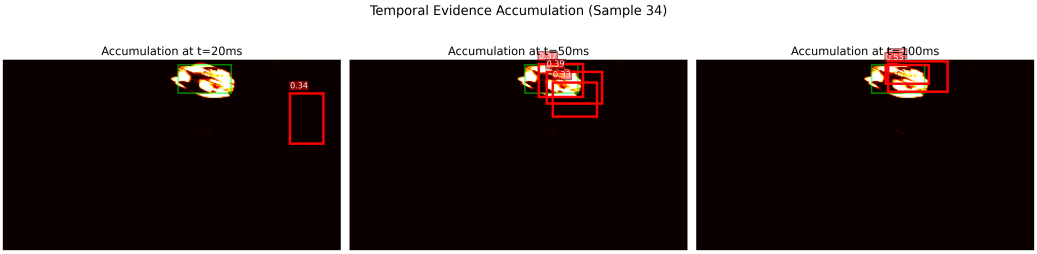
3.8.6 Temporal Evidence Accumulation

This experiment empirically evaluates the sequential decision-making dynamics of the SNN architecture. By extracting predictive snapshots at $t = 2$ (20ms), $t = 5$ (50ms), and $t = 10$ (100ms), we can observe the internal state evolution as the network integrates incoming spikes into a stable bounding box regression. This experiment serves to validate the biological plausibility of the model; specifically, its ability to perform rapid

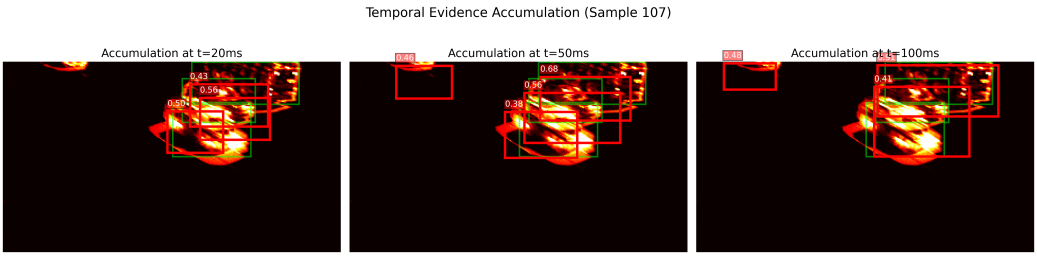
categorisation and sharpen its spatial belief over time.

Qualitative Evidence: Bounding Box Sharpening

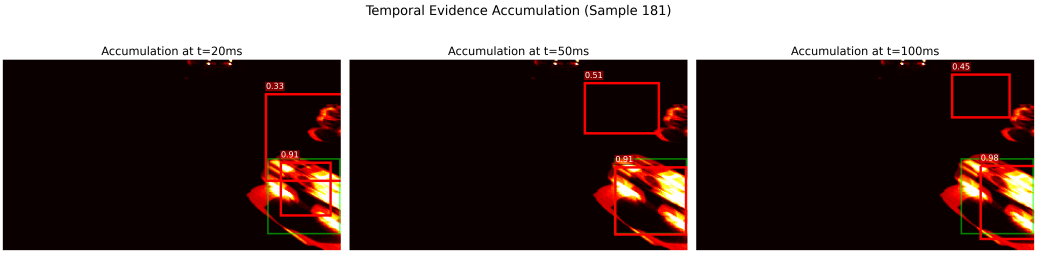
Figure 3.9 illustrates the predictive evolution for three representative validation samples across three snapshots at $t = 2$ (20ms), $t = 5$ (50ms), and $t = 10$ (100ms).



(a) Temporal Sequence 1: Initial hunting behavior and edge consolidation.



(b) Temporal Sequence 2: Persistent hunting with partial sharpening.



(c) Temporal Sequence 3: High-confidence early detection.

Figure 3.9: Evolution of SNN bounding box predictions over $T = 10$ timesteps, with annotated confidence scores. Each row demonstrates the progression from sparse initial evidence to a stable, high-confidence detection.

Critical Analysis

As established in the hypothesis (Section 3.7.4), the early-stage snapshots ($t = 2$) exhibit high coordinate jitter and lower confidence, as the LIF neurons have only begun to

aggregate sufficient membrane potential to overcome the firing threshold. Each sample in Figure 3.9 exhibit different behaviours:

Hunting Behaviour: Figure 3.9c most clearly demonstrates this phenomenon. At $t = 20$ms, multiple overlapping predictions scatter across a dense event cluster with moderate confidences (0.43–0.56), indicating early spike activity is sufficient to localise the scene region but insufficient to resolve individual vehicle boundaries. At $t = 50$ms, a spurious detection emerges with no underlying event support, characteristic of the regression head briefly committing to a spatially incorrect region as membrane potentials oscillate. By $t = 100$ms, partial consolidation occurs, and the model successfully observes the car that is partially visible in the top left corner, yet overlapping boxes and non-monotonic confidence scores persist. This behaviour is consistent with competing spike trains from multiple event sources (i.e. cars) preventing stable threshold-crossing conensus, meaning that the bounding boxes never truly settle — hunting without resolution. It can be attributed to the scene being too spatially complex for the network to disentangle, highlighting a limitation of this current architecture, and showing an area for future work to increase its complexity to handle such scenes more successfully.

Bounding Box Sharpening: Figure 3.9a best shows this phenomenon: at $t = 2$, the initial guess is nowhere near the ground truth, since the majority of LIF neurons have not accumulated sufficient membrane potential to cross the firing threshold ($V_{th} = 0.2$). However, as time progresses, the model correctly starts to localise on the car, and by $t = 10$ settles down into a stable prediction with a confidence of 53%. This sample in particular shows an area for further development, where a post-detection step could be introduced to accumulate nested predictions into one large prediction, which would prevent phenomena such as the one observed here: where there are two nested bounding boxes being drawn for the same event.

High-confidence early detection: Figure 3.9c shows that the current architecture succeeds in high-confidence early detection, correctly detecting the car in the bottom right corner with a confidence of 91% at the first timestep $t = 2$. In the context of vehicle detection which is time-sensitive, this is a significant finding because it proves that the model can support low-latency prediction accurately.

3.8.7 Real-Time Latency Benchmark

The real-time throughput of the model was quantified in FPS, derived from the reciprocal of the mean inference latency measured over 1,000 trials. To simulate a real-world edge-deployment scenario, a batch size of one was utilized, ensuring that the throughput reflects the model's ability to process individual temporal windows sequentially. Inference timing was performed using NVIDIA's asynchronous CUDA events (*torch.cuda.Event*) with explicit hardware synchronization to ensure that the reported FPS represents the raw computational latency of the GPU kernels, independent of CPU-bound communication overhead.

Results

Table 3.12 shows the difference in both latency and FPS between the CNN and SNN architectures. Beyond the mean throughput, the P99 latency (99th percentile) was measured to evaluate the temporal stability of the detectors. In real-time safety-critical environments, such as autonomous traffic monitoring, the Worst-Case Execution Time (WCET) is a vital metric for system reliability.

Metric	**SNN**	**CNN**
Mean Latency (ms)	3.8491	0.7130
Median Latency (ms)	3.8420	0.7086
Std Dev (ms)	0.0640	0.1163
P99 Latency (ms)	4.0930	0.7260
Throughput (FPS)	259.80	1402.60

Table 3.12: Inference latency benchmark results for the SNN and CNN baseline, measured with a pool size of 8. P99 denotes the 99th percentile latency.

Critical Analysis

The empirical results confirm the hardware-mismatch penalty hypothesized in Section 3.8.7: the CNN maintains a dominant speed advantage, outperforming the SNN by a factor of 5.4× (3.85ms vs. 0.71ms mean latency). This divergence is the direct result of simulating iterative LIF dynamics across $T = 10$ sequential timesteps on synchronous GPU hardware, which is inherently optimized for the single-pass dense matrix multiplications of the CNN paradigm.

Critically, however, the SNN remains well within the requirements for real-time

deployment. At 259.80 FPS, the spiking model processes high-definition data significantly faster than the 30 FPS threshold typically required for automotive perception [19]. This suggests that while the SNN faces a latency penalty on GPUs, the use of `step_mode='m'` parallelization mitigates the bottleneck enough to ensure the model remains viable for high-speed edge tasks, even when ran on conventional hardware.

Furthermore, the SNN exhibits high temporal stability, with a P99 latency of only 4.09ms (just 6.3% above the mean). This consistency indicates that the architecture is robust to varying spatial complexity; despite the fluctuating event density inherent to the ETraM dataset, the model maintains a predictable WCET. This ensures that high-traffic scenes do not induce hazardous latency spikes, validating the reliability of the spiking paradigm for the safety-critical feedback loops required in autonomous vehicle monitoring.

Chapter 4

Summary & Conclusions

4.1 Summary of Outcomes

This project has systematically deconstructed the performance-efficiency frontier of Spiking Neural Networks (SNNs) for high-definition automotive object detection. By evaluating the `DetectorNX ULTRA` architecture against a matched continuous-valued CNN baseline on the ETraM dataset [38], several definitive outcomes have been established that validate the neuromorphic paradigm for safety-critical edge deployment.

4.1.1 Global Precision Parity

The primary outcome of the spatial evaluation was the achievement of a mIoU of 0.4613 for the spiking model, representing 94.9% of the CNN baseline (0.4861). Furthermore, both architectures achieved a perfect recall of 100.0% for objects with an IoU > 0.5. These results prove that the architectural refinements introduced in this work—specifically the multi-box architecture, SE and SEW residual blocks—successfully mitigated the precision loss inherent in 1-bit quantization and discrete temporal resolution.

4.1.2 Decisive Efficiency Gains

The transition from dense CNN convolutions to event-driven spiking dynamics yielded an average power saving of 94.33% across the validation subset, which was driven by a 3.45$\times$ reduction in the total operations required to process a high-definition frame (9.22 G-MACs vs. 2.67 G-SOPs). Crucially, the SNN demonstrated "Reactive Scalability" ($R^2 = 0.7514$), where power consumption scaled linearly with scene complexity

rather than model size, a property that positions neuromorphic perception as inherently suited to the intermittent, sparse event streams characteristic of real-world automotive environments.

4.1.3 Temporal Reliability and Sharpening

Beyond global averages, the spiking paradigm demonstrated a unique "Bounding Box Sharpening" effect. As temporal evidence accumulated across $T = 10$ timesteps, the SNN was observed to produce more stable and spatially accurate regressions in high-traffic scenes compared to the single-pass CNN. This temporal integration capability allowed the SNN to exhibit a more decisive confidence distribution, being confident enough to classify noise with a confidence of 0%, effectively suppressing low-evidence background noise.

4.1.4 Real-Time Viability

Despite the hardware-mismatch penalty of executing iterative spiking logic on synchronous GPU hardware, the SNN achieved a mean inference latency of 3.85 ms (259.80 FPS). When evaluated against the industry-standard threshold for real-time automotive perception of 30 FPS [19], the spiking model maintains an 8.6× safety margin, despite using simulated LIF nodes. This validates the immediate viability of the proposed architecture for live deployment in vehicle-to-everything (V2X) and driver assistance systems.

In summary, the empirical evidence demonstrates that the `DetectorNX ULTRA` has successfully bridged the precision gap, establishing the spiking paradigm not merely as a theoretical curiosity, but as a robust, first-class solution for high-definition, power-constrained automotive perception.

4.2 Critical Reflection & Future Work

In order to appropriately contextualise the empirical findings, this section provides a critical analysis of the technical trade-offs and methodological limitations encountered during the development of the `DetectorNX` project. Paired with the critical analysis, we propose potential ideas in tandem that can be investigated to resolve these impacts.

While the results validate the efficiency of the spiking paradigm, several architectural and environmental constraints persist that define the frontier of current SNN research.

4.2.1 Hardware-Paradigm Inequity: The LIF Simulation Penalty

Impact

The most significant limitation of this evaluation is the hardware-mismatch penalty inherent in benchmarking an event-driven SNN on synchronous GPU hardware. Utilizing the SpikingJelly library's multi-step execution paradigm (`step_mode='m'`) [10], the $T = 10$ temporal window is processed as a series of fused-kernel iterations. The GPU must explicitly update the membrane potential and reset state for every neuron in every layer, regardless of whether a spike is generated.

This "simulated time" approach nullifies the asynchronous, event-driven nature of the SNN, resulting in the $5.4\times$ throughput penalty documented in Section 3.8.7.

Future Work

To achieve a truly definitive comparison, the architecture should be deployed on non-von Neumann silicon such as the IBM TrueNorth [22] or Intel Loihi [4]; only on such hardware can the 94.33% power savings be realized in a live, asynchronous context.

4.2.2 The Temporal Quantization Ceiling

Impact

The adoption of a fixed temporal window of $T = 10$ timesteps established a global precision ceiling that was particularly apparent in the quantization error analysis (Section 3.8.4). As established in early spiking vision literature, rate coding over discrete steps creates a $1/T$ (10%) quantization error in the feature maps [34]. This representational limit prevents the SNN from achieving the infinite sub-pixel coordinate refinement of the continuous CNN baseline.

Future Work

To break this fixed-precision trade-off, future trials should explore the following approaches:

- **Adaptive Timestepping:** A technique where the model dynamically increases temporal resolution for complex objects.
- **Time-to-First-Spike (TTFS):** An encoding method that utilizes exact spike timing; information is represented by the arrival time of the first spike rather than the firing frequency [2].
- **Rank Order Coding (ROC):** An approach that represents intensities via relative firing orders and sub-millisecond arrival times rather than discrete spike counts [34].

4.2.3 The Detection Head Mismatch

Impact

To enforce strict architectural parity and variable isolation, this project utilized a standard continuous-valued detection head for both models. While this ensured a clean empirical comparison of the backbones, it created an output stability gap. The observed "Hunting Behaviour" (Section 3.7.4) is a positive indicator of the SNN's high-frequency temporal sensitivity, proving the backbone is successfully reflecting the sub-millisecond jitter of the raw event stream.

However, the static, float-based head was too slow to resolve this highly reactive evidence, leading to the unstable bounding boxes observed in complex scenes. The use of a shared head was a deliberate methodological constraint to prevent the introduction of a "black box" variable, but it highlights the need for an SNN-native detection head, such as a membrane-regression head, to stabilize the model's sensitive temporal probing into a smoothed output suitable for ADAS and other applications in this domain.

Future Work

To bridge this output stability gap, future development must focus on the engineering of an SNN-native temporal fusion head. Rather than relying on a static float-based regression of discrete spikes, this novel head would regress bounding box coordinates continuously from the membrane potential (V_m) of the final layer. Alternatively, confidence logits could be derived directly from the relative firing rates of the output grid. This approach would natively interpret the temporal oscillation of the spiking backbone, smoothing the highly reactive event evidence into stable, monotonic predictions suitable for deployment in ADAS-like systems.

4.2.4 Dataset Domain Gap and Strategic Selection

Impact

Finally, the choice of the ETraM dataset [38] introduces a significant domain gap. Because the frames are captured from a static roadside perspective ($\sim$6 m height), they lack the ego-motion artifacts, such as motion blur and rapid optic flow, that define a vehicle-mounted sensor's environment. In real-world autonomous deployments, cameras are mounted directly to moving vehicles; therefore, it is imperative that a perception system is trained to process and filter such dynamic artifacts.

However, the selection of ETraM over alternatives like Prophesee GEN1 was a deliberate methodological trade-off; ETraM was prioritized for its manual label fidelity (human-assigned labels) and HD resolution, providing a gold standard benchmark that was essential for the architectural validation of the `ULTRA` backbone. While the static mounting is a weakness, the quality of the annotations ensured that the 94.9% precision parity was a reflection of the architecture's capacity, rather than an artifact of noisy, automated labels. Furthermore, the ETraM study also provided a custom pre-processing pipeline optimized towards the specific dataset [38], which served as the foundation for the `REPLICA` pipeline detailed in Section 3.3. Leveraging this existing infrastructure significantly accelerated development, ensuring the project's ambitious scope could be realized within the constrained timeframe.

Future Work

To evolve `DetectorNX` from a static benchmark into a production-ready automotive system, future evaluations must incorporate vehicle-mounted datasets to expose the model to ego-motion artifacts. To maintain the high-fidelity representation learned from ETraM, future work should employ a joint-training or domain-adaptation strategy. The architecture could be pre-trained on the gold-standard static labels of ETraM and subsequently fine-tuned on a meticulously manually-relabeled subset of the Prophesee GEN1 dataset [36]. This hybrid approach would validate and train the architecture's robustness against severe optic flow and dynamic backgrounds while circumventing the label noise inherent to fully automated RGB-to-event annotations.

Furthermore, the ultimate test of the SNN's low-latency advantage requires integration into a closed-loop automotive simulator such as CARLA [7] equipped with simulated event cameras. This would shift the evaluation metric from static precision (mIoU) to physical safety outcomes, definitively quantifying how the SNN's 3.85 ms

inference latency reduces emergency braking distances in dynamic environments.

Bibliography

[1] Peter Blouw et al. 'Benchmarking Keyword Spotting Efficiency on Neuromorphic Hardware'. In: *Proceedings of the 7th Annual Neuro-inspired Computational Elements Workshop (ICONS)*. 2019, pp. 1–8. DOI: 10.1145/3320288.3320304.

[2] Lina Bonilla et al. 'Analyzing time-to-first-spike coding schemes: A theoretical approach'. In: *Frontiers in Neuroscience* Volume 16 - 2022 (2022). ISSN: 1662-453X. DOI: 10.3389/fnins.2022.971937. URL: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2022.971937.

[3] Loïc Cordone, Benoît Miramond and Philippe Thierion. *Object Detection with Spiking Neural Networks on Automotive Event Data*. 2022. arXiv: 2205.04339 [cs.CV]. URL: https://arxiv.org/abs/2205.04339.

[4] Mike Davies et al. 'Loihi: A Neuromorphic Manycore Processor with On-Chip Learning'. In: *IEEE Micro* 38.1 (2018), pp. 82–99. DOI: 10.1109/MM.2018.112130359.

[5] Lei Deng et al. 'Comprehensive SNN Compression Using ADMM Optimization and Activity Regularization'. In: *IEEE Transactions on Neural Networks and Learning Systems* 34.6 (2023), pp. 2791–2805. DOI: 10.1109/TNNLS.2021.3109064.

[6] Michele Deodato and David Melcher. 'Continuous temporal integration in the human visual system'. In: *Journal of Vision* 24.13 (Dec. 2024), pp. 5–5. ISSN: 1534-7362. DOI: 10.1167/jov.24.13.5. eprint: https://arvojournals.org/arvo/content_public/journal/jov/938697/i1534-7362-24-13-5_1733390407.54037.pdf. URL: https://doi.org/10.1167/jov.24.13.5.

[7] Alexey Dosovitskiy et al. 'CARLA: An Open Urban Driving Simulator'. In: *CoRR* abs/1711.03938 (2017). arXiv: 1711.03938. URL: http://arxiv.org/abs/1711.03938.

[8] Jason K Eshraghian et al. 'Training Spiking Neural Networks Using Lessons From Deep Learning'. In: *Proceedings of the IEEE* 111.9 (2023), pp. 1016–1054. DOI: 10.1109/JPROC.2023.3308088.

[9] Wei Fang et al. *Deep Residual Learning in Spiking Neural Networks*. 2022. arXiv: 2102.04159 [cs.NE]. URL: https://arxiv.org/abs/2102.04159.

[10] Wei Fang et al. *SpikingJelly: An open-source machine learning infrastructure platform for spike-based intelligence*. 2023. arXiv: 2310.16620 [cs.NE]. URL: https://arxiv.org/abs/2310.16620.

[11] Guillermo Gallego et al. 'Event-Based Vision: A Survey'. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.1 (2022), pp. 154–180. DOI: 10.1109/TPAMI.2020.3008413.

[12] Daniel Gehrig et al. *End-to-End Learning of Representations for Asynchronous Event-Based Data*. 2019. arXiv: 1904.08245 [cs.CV]. URL: https://arxiv.org/abs/1904.08245.

[13] Chuan Guo et al. 'On calibration of modern neural networks'. In: *International conference on machine learning*. PMLR. 2017, pp. 1321–1330.

[14] Kaiming He et al. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385 [cs.CV]. URL: https://arxiv.org/abs/1512.03385.

[15] Mark Horowitz. '1.1 computing's energy problem (and what we can do about it)'. In: *2014 IEEE international solid-state circuits conference digest of technical papers (ISSCC)*. IEEE. 2014, pp. 10–14.

[16] Jie Hu et al. *Squeeze-and-Excitation Networks*. 2019. arXiv: 1709.01507 [cs.CV]. URL: https://arxiv.org/abs/1709.01507.

[17] Cristina Iaboni and Pramod Abichandani. *Event-based Spiking Neural Networks for Object Detection: A Review of Datasets, Architectures, Learning Rules, and Implementation*. 2024. arXiv: 2411.17006 [cs.NE].

[18] Glenn Jocher, Ayush Chaurasia and Jing Qiu. *Ultralytics YOLOv8*. Version 8.0.0. 2023. URL: https://github.com/ultralytics/ultralytics.

[19] Abdul Hannan Khan, Syed Tahseen Raza Rizvi and Andreas Dengel. 'Real-time traffic object detection for autonomous driving'. In: *arXiv preprint arXiv:2402.00128* (2024).

[20] Yann LeCun, Yoshua Bengio and Geoffrey Hinton. 'Deep learning'. In: *Nature* 521.7553 (2015), pp. 436–444. DOI: 10.1038/nature14539. URL: https://doi.org/10.1038/nature14539.

[21] Shih-Chieh Lin et al. 'The Architectural Implications of Autonomous Driving: Constraints and Acceleration'. In: *SIGPLAN Not.* 53.2 (Mar. 2018), pp. 751–766. ISSN: 0362-1340. DOI: 10.1145/3296957.3173191. URL: https://doi.org/10.1145/3296957.3173191.

[22] Paul A Merolla et al. 'A million spiking-neuron integrated circuit with a scalable communication network and interface'. In: *Science* 345.6197 (2014), pp. 668–673. DOI: 10.1126/science.1254642.

[23] Emre O Neftci, Hesham Mostafa and Friedemann Zenke. 'Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-Based Optimization to Spiking Neural Networks'. In: *IEEE Signal Processing Magazine* 36.6 (2019), pp. 51–63. DOI: 10.1109/MSP.2019.2931595.

[24] Emre O. Neftci, Hesham Mostafa and Friedemann Zenke. 'Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-Based Optimization to Spiking Neural Networks'. In: *IEEE Signal Processing Magazine* 36.6 (2019), pp. 51–63. DOI: 10.1109/MSP.2019.2931595.

[25] Michael Pfeiffer and Thomas Pfeil. 'Deep Learning With Spiking Neurons: Opportunities and Challenges'. In: *Frontiers in Neuroscience* Volume 12 - 2018 (2018). ISSN: 1662-453X. DOI: 10.3389/fnins.2018.00774. URL: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2018.00774.

[26] Joseph Redmon et al. 'You only look once: Unified, real-time object detection'. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 779–788.

[27] Hamid Rezatofighi et al. 'Generalized Intersection Over Union: A Metric and a Loss Function'. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 658–666. DOI: 10.1109/CVPR.2019.00075.

[28] Kaushik Roy, Akhilesh Jaiswal and Priyadarshini Panda. 'Towards spike-based machine intelligence with neuromorphic computing'. In: *Nature* 575.7784 (Nov. 2019), pp. 607–617. ISSN: 1476-4687. DOI: 10.1038/s41586-019-1677-2. URL: https://pubmed.ncbi.nlm.nih.gov/31776490/.

[29] Kaushik Roy, Akhilesh Jaiswal and Priyadarshini Panda. 'Towards spike-based machine intelligence with neuromorphic computing'. In: *Nature* 575.7784 (2019), pp. 607–617.

[30] Rahul Sarpeshkar, Lloyd Watts and Carver Mead. 'Refractory neuron circuits'. In: (1992).

[31] Abhronil Sengupta et al. *Going Deeper in Spiking Neural Networks: VGG and Residual Architectures*. 2019. arXiv: 1802.02627 [cs.CV]. URL: https://arxiv.org/abs/1802.02627.

[32] Nicolas Skatchkovsky, Hyeryung Jang and Osvaldo Simeone. 'Bayesian Continual Learning via Spiking Neural Networks'. In: *Frontiers in Computational Neuroscience* 16 (2022), p. 1037976. DOI: 10.3389/fncom.2022.1037976.

[33] Zhe Su et al. *Core interface optimization for multi-core neuromorphic processors*. 2023. arXiv: 2308.04171 [cs.AR]. URL: https://arxiv.org/abs/2308.04171.

[34] Simon Thorpe, Arnaud Delorme and Rufin Van Rullen. 'Spike-based strategies for rapid processing'. In: *Neural Networks* 14.6 (2001), pp. 715–725. ISSN: 0893-6080. DOI: https://doi.org/10.1016/S0893-6080(01)00083-1. URL: https://www.sciencedirect.com/science/article/pii/S0893608001000831.

[35] Simon Thorpe, Denis Fize and Catherine Marlot. 'Speed of processing in the human visual system'. In: *Nature* 381.6582 (1996), pp. 520–522. DOI: 10.1038/381520a0.

[36] Pierre de Tournemire et al. 'A Large Scale Event-based Detection Dataset for Automotive'. In: *CoRR* abs/2001.08499 (2020). arXiv: 2001.08499. URL: https://arxiv.org/abs/2001.08499.

[37] Khanin Udomchoksakul, Orathai Sangpetch and Akkarit Sangpetch. 'GPU Performance Tuning and Power Efficiency on the DGX A100 Cluster'. In: *2022 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. 2022, pp. 170–177. DOI: 10.1109/CloudCom55334.2022.00033.

[38] Aayush Atul Verma et al. ‘eTraM: Event-based Traffic Monitoring Dataset’. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2024, pp. 22637–22646.

[39] Yulong Yan et al. ‘Backpropagation With Sparsity Regularization for Spiking Neural Network Learning’. In: *Frontiers in Neuroscience* 16 (2022), p. 760298. DOI: 10.3389/fnins.2022.760298.

Remove duplicate references.

Move ArXiv to peer-reviewed sources.